

PART I · 基础

PART II · Skills

PART III · Subagents

PART IV · Hooks

PART V · 组合

PART VI + 附录

本 PART 章节

19. Ch 19 · The Three-Pillar Architecture

20. Ch 20 · Case Study: Full-Stack Development Environment

21. Ch 21 · Case Study: The Solo Technical Founder's Setup

原书第 179 页 · PART FIVE

179

PART FIVE · 第五部分

The Composition Layer: Building a Self-Governing Development Environment · 组合层:搭一个"自我治理"的开发环境

Three pillars, one architecture. The whole begins to mean more than the sum of the parts.

三根支柱,一套架构。整体开始意味着"比各部分之和更多"的东西。

CHAPTER NINETEEN · 第十九章

The Three-Pillar Architecture: Designing a Complete Claude Code System · 三支柱架构:设计一套完整的 Claude Code 系统

← 前一页

后一页 →

原书第 180 页 · CHAPTER NINETEEN

180

"Architecture is the art of arranging elements so that the whole has properties none of the elements possess. Without arrangement, there is only inventory."

— Christopher Alexander, paraphrased

"所谓架构,就是把元素安排成一种形态,使整体具有任何单一元素都不具备的性质。没有安排,就只剩一份清单。"

—— 克里斯托弗·亚历山大(意译)

Up to this point in the book, the three pillars have been treated separately. Skills were the subject of Part Two, subagents the subject of Part Three, hooks the subject of Part Four. Each part developed its pillar in depth, with mechanics, design patterns, and a working library installed by the chapter's end. The treatment was deliberately separate, because each pillar carries its own design discipline and learning each one in isolation is the only way to develop the discipline cleanly. But the separation is now over. The remainder of the book treats the three pillars as components of a single architecture, and this chapter is where the architecture itself gets explicit attention.

到本书此前的位置为止,三根支柱是分开讲的。Skill 是第二部分的主题,subagent 是第三部分的主题,hook 是第四部分的主题。每一部分都把各自那根支柱讲得很深——含机制、设计模式,以及"到章末装好的一套能跑起来的库"。这种分开是有意为之——因为每根支柱都自有一套设计纪律,而"一门一门分开学",是把纪律练得干净的唯一办法。但"分开"这件事,到此为止。书剩下的部分,会把三根支柱当作一套架构里的若干组件来对待;而本章,正是让这套架构本身"被显式注意"的地方。

The argument of this chapter is that any sophisticated Claude Code environment uses all three pillars, that the pillars have a natural division of responsibility, that the composition between them follows recognizable patterns, and that the resulting environment has properties that no single pillar could produce on its own. The architecture is not arbitrary. It reflects the structure of the tool, the structure of real engineering work, and the structure of the failure modes that the architecture is built to prevent. Once the reader has internalized the architecture, the design of any new Claude Code environment becomes a matter of applying the same patterns to her specific work, with the patterns themselves as a stable starting point.

本章的论点是:任何一套"够复杂"的 Claude Code 环境,都会同时用上三根支柱;三根支柱之间存在一种自然的职责划分;它们彼此组合时,会遵循若干可识别的模式;而组合出来的环境,具有"任一根支柱单兵作战"都达不到的性质。这套架构并非随意——它反映的是工具本身的结构、真实工程工作的结构、以及"这套架构要去防住"的那几类失效模式的结构。一旦读者把架构内化,设计任何新的 Claude Code 环境,就变成"把这套同样的模式套到自己的具体工作上";模式本身,则成为一个稳定的起点。

Begin with the division of responsibility. Each pillar handles a class of problem that the other two cannot handle as well. Skills handle capability; they teach Claude how to do specific kinds of work consistently. Subagents handle specialization and parallelism; they give specific kinds of work their own focused context, their own tuned role, and the option to run alongside other work simultaneously. Hooks handle determinism; they impose mechanical guarantees on the lifecycle, ensuring that what must happen does happen, and what must not happen cannot happen. The three together cover capability, specialization, and boundary, which are the three categories of concern in any serious workflow.

从"职责划分"讲起。每一根支柱,各自处理一类"另两根处理得没那么好"的问题。Skill 处理"能力"——教 Claude 如何把某类工作一致地做对。Subagent 处理"专门化"与"并行"——为某类工作配上聚焦的上下文、调好的角色,以及"和其他工作并行跑"的可能性。Hook 处理"确定性"——对生命周期施加机械性的保证,让"必须发生的"确实发生,"绝不能发生的"绝不能发生。三者合起来,覆盖了"能力 / 专门化 / 边界"这三类——它们正是任何严肃工作流都绕不开的三个关切类别。

← 前一页

(下一批翻译) →

A simple way to remember the division is by the question each pillar answers. Skills answer the question, how should this be done. Subagents answer the question, who should do this. Hooks answer the question, what is permitted. A workflow that uses only skills can teach Claude to do good work but has no way to isolate different kinds of work and no way to guarantee enforcement. A workflow that uses only subagents has specialization but no stored knowledge and no enforcement. A workflow that uses only hooks has enforcement but no capability and no specialization. Each pillar, alone, is a partial answer. All three, together, form a complete one.

记住三者分工的一个简单办法,就是看每一柱回答的是哪个问题。Skills 回答的是"这件事该怎么做"。Subagents 回答的是"这件事该由谁做"。Hooks 回答的是"这件事什么被允许做"。只装 skills 的工作流,能教会 Claude 把活做漂亮,但既无法把不同类的工作隔离开来,也无法保证规约被执行。只装 subagents 的工作流,有专门化但没有沉淀下来的知识,也没有强制力。只装 hooks 的工作流,有强制力但没有能力,也没有专门化。任一柱单独存在,都只是一个"局部解";三柱一起,才构成一个完整的答案。

Two Layers of the Architecture · 架构的两层

The next observation, which is more subtle, is that the three pillars sit at different layers of the architecture. Skills and subagents both operate inside the agent loop. They are read by the model. They shape the model's behavior through prose. They live in the probabilistic space where the model interprets, reasons, and decides. Hooks operate outside the loop. They run as shell commands, not as language. They are not interpreted by the model. They are mechanical. The two layers, probabilistic inside and deterministic outside, are the natural partitioning of responsibility in a well-designed environment. The model is given freedom and capability inside the loop. The boundaries of the loop are deterministic. This shape, what one practitioner has called the deterministic outer and the probabilistic inner, is the central architectural pattern of the book, and it is worth holding in working memory because it explains almost every design decision in any complete environment.

下一条观察更微妙一些:三柱其实处在架构的不同层面上。Skills 和 subagents 都是在 agent loop 内部工作的——它们由模型读取,通过对模型吐出的文字(prose)塑形;它们栖身在那个"模型在其中解读、推理、做决定"的概率性空间里。Hooks 则是在 loop 外部工作的——它们以 shell 命令的形式运行,不是语言,也不会被模型"解读";它们是机械的。这两层——内部概率性、外部确定性——正是一个设计良好的环境里"职责"最自然的切分方式。模型在 loop 内被给予自由和能力;loop 的边界则是确定性的。这个形状(有位实践者称之为"外层确定、内层概率")是本书的中心架构范式,值得放在工作记忆里——因为它能解释任何一个完整环境里几乎所有的设计决策。

A worked illustration brings the architecture to life. Consider a developer who needs to perform a code review. The work has several components. The reviewer needs to know how good code reviews are written in this codebase. The reviewer needs to apply different kinds of attention to different aspects of the change. The reviewer needs to be prevented from accidentally approving changes that touch protected paths. Each of these components maps cleanly to one of the three pillars. The how is encoded in skills: a project-level `pr-review` skill that knows the team's preferred review format. The who is encoded in subagents: parallel security, performance, and architecture specialists who each handle their dimension.

举一个落地的例子,可以让这套架构活起来。想象一位开发者要执行一次 code review。这项工作包含好几个组件:reviewer 需要知道"在这个代码库里,一份好的 code review 应当怎么写";reviewer 需要对变更的不同方面施加不同种类的注意力;reviewer 还需要被防止在"被改的文件刚好落在受保护路径上"时不小心点了 approve。这每一个组件,都能干净地映射到三柱的其中一柱上。"怎么做"编码在 skill 里:一个 project 级别的 `pr-review` skill,知道本团队偏好的评审格式。"由谁做"编码在 subagent 里:并行运行的安全、性能、架构专员,各自处理自己那一维。

← 前一页

后一页 →

The *what-is-permitted* is encoded in hooks: a gate on `PreToolUse` that blocks any tool call that would mark a PR approved without explicit human confirmation when the PR touches authentication code.

"什么被允许"编码在 hooks 里:一道挂在 `PreToolUse` 上的 gate,会在"PR 触及身份认证相关代码、且没有任何人工确认时"拦截任何把它标为 approved 的工具调用。

The composition runs naturally. The developer asks for a review. Skills fire because the request matches their descriptions. Subagents are dispatched in parallel because the orchestrator pattern recognizes the multidimensional shape of the work. Hooks watch the lifecycle silently. The review unfolds in two minutes, the unified result is presented to the developer, and the deterministic guarantees protect the boundary that the model alone could not be trusted to respect. None of this required the developer to think about the architecture during the request. She typed a sentence. The architecture did the work.

三柱的组合自然地跑起来。开发者请求一次评审;skill 因为请求匹配了 description 而触发;subagent 因为 orchestrator 范式识别出了"这件事是多个维度"而被并行派出去;hooks 在生命周期里静默地守着。两分钟后,review 跑完,统一的结果呈到开发者面前,确定性的那条边界——模型本身无法被信赖会去守的那条——被 hooks 守住了。整件事,并不要求开发者在请求时心里还惦记着这套架构。她只敲了一句话,架构就替她把活干了。

A second observation about the architecture concerns the order of construction. The natural order in which a developer should build her environment is the order in which the pillars were taught in this book: skills first, subagents second, hooks third. The order is not arbitrary. Skills are the broadest and most generally useful pillar; they pay back immediately in almost every session, and they are the easiest to write well on the first attempt. Subagents are the next layer; they require slightly more design effort but produce dramatic benefits when the work has the right shape. Hooks are the most specialized; their value is concentrated in specific safety-critical concerns and in workflow enforcement, and they require the most careful design to install correctly. Building in this order means that the early investment produces immediate benefits, the middle investment produces architectural benefits, and the late investment produces the deterministic guarantees that complete the system. A developer who tries to build in reverse order, hooks first and skills last, often finds the experience frustrating, because hooks without skills and subagents are just constraints on a vanilla environment, and the constraints feel restrictive rather than enabling.

关于架构的第二条观察,是关于"建造顺序"的。开发者搭建她环境时,自然的顺序,就是本书讲述这三柱的顺序:先 skills,再 subagents,最后 hooks。这个顺序不是随意的。Skills 是覆盖面最广、最普适的那一柱;它几乎在每一个 session 里都能立刻见回报,而且第一次写就写得不错的难度也最低。Subagents 是下一层;它需要稍微多花点设计力气,但只要工作形状对得上,带来的收益是戏剧性的。Hooks 则是最专门化的;它的价值集中在某些"安全攸关"的具体关切,以及 workflow 强制,装的时候也需要最仔细的设计。按这个顺序搭,意味着前期的投入立刻就见到效益,中期的投入换来架构层面的效益,后期的投入换来"让系统真正闭环"的那些确定性保证。要是反过来——先装 hooks,最后才装 skills——体验常常很令人沮丧:没有 skills 和 subagents 的 hooks,只是套在一个"素面"环境上的约束,这种约束会让人感到被束缚,而不是被赋能。

Failure Modes Prevented · 架构所规避的失效模式

A third observation, this one about the failure modes the architecture prevents. Each pillar prevents a different category of failure. Skills prevent the failure of inconsistency: without skills, the developer's own attention has to carry every convention, and her attention is fallible. Subagents

第三条观察,是关于"这套架构规避了哪些失效模式"。每一柱规避的是不同类别的失效。Skills 规避的是"不一致"这种失效:没有 skills,每一个规约都得靠开发者自己的注意力去扛,而人的注意力是会出错的。Subagents

prevent the failure of context pollution: without subagents, the main session's context fills with the residue of every phase of work, and the quality of every phase suffers. Hooks prevent the failure of probabilistic breach: without hooks, the model's judgment is the only line of defense

against catastrophic mistakes, and the model's judgment is non-zero failure rate. The architecture, taken as a whole, is a defense in depth against the three most common ways that AI-assisted workflows go wrong. A developer who has installed all three pillars has, in effect, hardened her workflow against the three failure modes that would otherwise haunt it.

规避的是"上下文污染"这种失效:没有 subagents,主会话的上下文会被每一阶段工作的残留物填满,每一阶段的质量都会跟着掉。Hooks 规避的是"概率性破防"这种失效:没有 hooks,模型自己的判断就成了抵御灾难性错误的唯一防线,而模型的判断是有一个非零失败率的。把三柱合起来看,这套架构对"AI 辅助工作流最常见的三种出错方式"形成了一道纵深防御。一个装好了三柱的开发者,事实上是在把她的工作流硬化,以抵御那三种若不这样就会始终缠着她的失效模式。

A fourth observation concerns the design process for adding new capabilities to the environment. When the developer notices a friction in her work that she wants to address, she should ask herself which pillar fits. The classification questions are simple. Is this a knowledge gap, where the developer wishes the environment knew something specific? That is a skill. Is this an attention gap, where the developer wishes a different kind of attention were applied to a kind of work? That is a subagent. Is this a guarantee gap, where the developer wishes that something would always or never happen? That is a hook. Asking the right question first prevents the most common design mistakes, which are building skills that should have been hooks, building hooks that should have been skills, and building subagents that should have been narrow skills inside a regular session. The classification is fast, the right answer is usually obvious once the classification has been made, and the cumulative effect of asking the question every time is an environment whose components live in the right places.

第四条观察,是关于"向环境里加新能力"的设计流程。当开发者在工作中留意到一处她想去消除的摩擦时,她应当先问自己"这该落到哪一柱上"。分类问题本身很简单:这是一个"知识缺口"吗——也就是你希望环境知道某件具体的事?那就是 skill。这是一个"注意力缺口"吗——也就是你希望"某类工作被施加一种不同种类的注意力"?那就是 subagent。这是一个"保证缺口"吗——也就是你希望某件事要么永远发生、要么永远不发生?那就是 hook。先问对问题,可以规避最常见的设计错误:把本该做成 hook 的事做成 skill,把本该做成 skill 的事做成 hook,把本该是"普通 session 里一个窄 skill"的事做成 subagent。分类很快,一旦分类完成,正确答案通常就一目了然;而每次都问一遍这个问题,长年累月下来,环境里的每一个组件都会落在该落的位置上。

A fifth observation concerns over-engineering. The temptation to build an elaborate environment is real, and most developers go through a phase where they install too many skills, too many subagents, too many hooks, and the environment becomes harder to maintain than the work it was supposed to support. The remedy is to start small, add components only when their need is concrete, and remove components that have not been used in a few months. A small, well-designed environment of fifteen skills, six subagents, and eight hooks is dramatically more useful than a sprawling one with fifty of each. The compounding of component count past a certain point becomes a tax rather than a benefit, because the developer has to keep more in her head, the failure modes multiply, and the cost of maintenance

第五条观察是关于"过度工程化"的。搭建一个庞大环境的诱惑是真实的;大多数开发者都会经历这么一个阶段——装了太多 skills、太多 subagents、太多 hooks,环境变得比它本该支持的工作还难维护。解法是:起步要小,只在需求具体且明确时才加新组件,几个月没用上的就清掉。一个精巧、设计良好的环境(15 个 skills、6 个 subagents、8 个 hooks)远比一个松散铺开的 50 个同款有用得多。组件数一旦越过某个临界点,就开始变成"税"而不是"福利"——因为开发者脑子里要装的东西变多,失效模式也成倍增加,而维护的成本,

← 前一页

后一页 →

starts to consume the benefit of the components themselves. The discipline of restraint is part of the architecture.

开始反噬组件本身带来的好处。"克制"这条纪律,本身也是这套架构的一部分。

A sixth observation concerns the maintenance posture of the environment. A well-built environment is not a static artifact. It evolves with the developer's work. New skills get added when new frictions emerge. Old skills get refined as the developer's preferences change. Subagents get reshaped as the developer learns which roles benefit most from isolation. Hooks get updated as new threats and conventions emerge. The `.claude` directory of any active developer should look noticeably different at the end of a year from how it looked at the start, because the developer's environment has been growing alongside her work. The maintenance investment is small; thirty minutes per week is sufficient to keep an environment current and useful. The compounding of that investment, across the years a developer uses her environment, is what produces the gap between developers who treat their tooling seriously and developers who do not.

第六条观察是关于“环境的维护姿态”。一个搭建得好的环境,不是一个静态的产物——它会跟着开发者的工作一起演化。新摩擦出现时,会有新 skill 加进来;开发者偏好变了,旧 skill 会被精修;在实践中摸清“哪些角色最受益于隔离”之后,subagent 会被重新塑形;新的威胁、新的规约浮现时,hook 会被更新。任何一位活跃开发者的 `.claude` 目录,在一年末尾的样子,都应当和年初的样子明显不同——因为她的环境,一直在和她的工作一起长大。维护的投入并不大:每周 30 分钟,就足以让一个环境保持鲜活、好用。而这份额不大的投入,在开发者使用她环境的这些年里不断复利,正是“认真对待自己工具的开发者和”不这么做的开发者”之间那条鸿沟的来源。

What This Architecture Means · 这套架构意味着什么

The final observation in this chapter is the one that matters most for the rest of the book. The composition principle, repeated throughout these chapters, is not just a design pattern. It is the difference between using Claude Code and engineering with it. A developer who has built a thoughtful three-pillar environment is doing something fundamentally different from a developer who has not. She is operating inside an environment that has been shaped to her work, that absorbs frictions she would otherwise carry, that protects her from mistakes she would otherwise make, and that produces a level of consistency and quality across her output that no amount of careful prompting alone could match. The investment to build this environment is real but bounded. The benefit, once built, compounds for as long as the developer continues to work in the environment. There is no comparable investment available to a software engineer in 2026 that produces anything close to the leverage of a well-designed Claude Code environment, because no other tool sits at the intersection of capability, specialization, and enforcement that Claude Code does.

本章的最后一条观察,是往后整本书里最关键的一条。在前面这些章节里反复出现的“组合原则”,不只是一个设计范式——它是“用 Claude Code”和“用 Claude Code 做工程”之间那条分水岭。一个搭建了深思熟虑的“三柱环境”的开发者,和一个没有这样做的开发者,做着本质不同的事。前者是在一个已经被塑造得贴合她工作的环境里运转:环境吸收她原本要自己扛的摩擦、保护她免于她原本会犯的错、并让她产出的一致性和质量达到“再仔细的 prompt 工程”也匹配不上的水平。搭建这个环境的投入是真实但有限的;一旦搭好,收益会在她继续使用环境的每一天里不断复利。在 2026 年,软件工程师能做的投资里,没有哪一项能在杠杆效率上接近“一个设计良好的 Claude Code 环境”——因为没有任何别的工具,同时坐在能力、专门化、强制这三件事的交汇点上,而 Claude Code 坐在那里。

The next two chapters take the architecture from principle to practice. Each chapter is a substantial case study in which a complete environment is

接下来两章,要把这套架构从原则带到实践。每一章都是一个分量十足的案例研究——会从头到尾为一个特定种类的工作,搭出一个完整的环境——

built end-to-end for a specific kind of work. The first case study, in the chapter that follows, is a full-stack development environment for a TypeScript engineer. The second, in the chapter after that, is a solopreneur's environment for a technical founder running product, marketing, and operations from a single Claude Code installation. Together, the two case studies demonstrate the architecture across two very different shapes of work, and they leave the reader with concrete blueprints she can adapt to her own situation. The architectural principles end here. The applied builds begin in the chapter ahead.

端到端地搭出来。第一个案例在紧接的这一章:为一位 TypeScript 工程师搭的全栈开发环境。第二个案例在再后面那一章:为一位独立创业者搭的、用于"从一份 Claude Code 安装里同时跑产品、营销、运营"的环境。两个案例合起来,把同一套架构展示在两种非常不同的工作形状上,也会给读者留下可以拿回去套自己情况的、具体可用的蓝图。架构原则到此为止;实战搭建,从下一章开始。

Before turning to the case studies, a few additional architectural patterns deserve explicit attention, because they show up in almost every well-built environment and the reader should recognize them when she sees them in the chapters ahead.

在转向案例之前,还有几条额外的架构范式值得明说——它们几乎会出现在每一个搭得好的环境里,读者应当能在接下来两章里一看到它们就认出来。

The first pattern is what might be called the project-skill, user-subagent split. In well-designed multi-developer environments, project-specific knowledge tends to live in skills at the project level, while role specialists tend to live as subagents at the user level. The reason is that knowledge is project-specific (a Python project has different conventions from a TypeScript project) but roles are usually personal (a developer's preferred way of approaching code review does not change much when she switches projects). The split allows the developer to maintain a stable personal subagent library that she carries from project to project, while each project contributes its own skills that augment her subagents with project-specific knowledge. The composition pattern from Chapter Fourteen depends on this split working cleanly, and developers who set up their environments with the split in mind from the start avoid a common refactoring later when they realize their subagents have accumulated project-specific knowledge that should have lived elsewhere.

第一条范式,可以叫做"项目级 skill、用户级 subagent"的拆分。在设计良好的多开发者环境里,"项目特有的知识"倾向于作为 skill 落在 project 级别;"角色专家"则倾向于作为 subagent 落在 user 级别。理由在于:知识是项目特有的(Python 项目和 TypeScript 项目的规约不一样),而角色通常是个人的(开发者偏好的 code review 方式,她换了项目也基本不变)。这种切分让开发者可以维护一个稳定的"个人 subagent 库",从项目带到项目;而每个项目自己贡献自己的 skill,在她的 subagent 之上"长出"项目特有的知识。第十四章讲过的"组合范式"依赖这种切分能干干净净地跑;从一开始就按这个切分来组织环境的开发者,会避免掉后来一种很常见的重构:她突然发现,自己的 subagent 不知道什么时候塞进了"本不该属于它"的项目特有知识,得重新拆出来。

Ambient Versus Targeted · 环境级 vs 触发级

The second pattern is the ambient-versus-targeted split, which was introduced in Chapter Two but deserves restatement in the architectural context. Information that is true at all times in a project belongs in `CLAUDE.md`, where it is loaded into every session's context unconditionally. Information that is true only when a specific kind of work is being done belongs in a skill, where it is loaded only when the skill activates. The line

第二条范式是"环境级 vs 触发级"的切分,第二章引入过,但放在架构的上下文里值得再讲一遍。一个项目里"任何时候都为真"的信息,放在 `CLAUDE.md` 里——它会在每个 session 的上下文里无条件被加载。"只在做某类特定工作时才为真"的信息,放在 skill 里——只有当这个 skill 被触发时,它才会被加载。这条线

between the two is sometimes blurry, but a useful rule is that any piece of knowledge that costs more than fifty words to express, and that is relevant to less than a quarter of the work in the project, almost certainly belongs in a skill rather than in `CLAUDE.md`. Putting too much in `CLAUDE.md` inflates the baseline context cost of every session and degrades the model's attention. Putting too little in `CLAUDE.md` makes the developer brief Claude on the same project facts every session, which defeats the purpose of having ambient context at all. The architectural discipline is to push as much as possible into skills and keep `CLAUDE.md` to genuinely ambient material.

有时候画得不太清,但有一条好用的经验:任何"表达起来超过 50 个字"、且"相关性覆盖不到项目四分之一工作"的知识块,几乎可以肯定该放在 skill 里,而不是 `CLAUDE.md` 里。`CLAUDE.md` 里塞太多,会撑大每个 session 的基线上下文成本,让模型的注意力变差; `CLAUDE.md` 里塞太少,又会让开发者每个 session 都要把同一组项目事实再向 Claude 复述一遍——这就完全失去了"环境级上下文"存在的意义。架构层面的纪律是:能推到 skill 里的就尽量推到 skill 里,把 `CLAUDE.md` 留给真正"环境级"的那些内容。

The third pattern is the hooks-as-the-last-resort principle. When the developer is considering whether to enforce some convention with a hook or with a skill, the right default is to start with a skill and only escalate to a hook if the skill is not enough. Skills are softer, more flexible, easier to maintain, and more conversational. Hooks are harder, more rigid, and harder to debug when they go wrong. A convention that the model can reliably honor when reminded should live in a skill. A convention that the model genuinely cannot be trusted to honor, where the consequences of a single failure are substantial, should live in a hook. This default keeps the architecture honest. The developer who reaches for hooks first ends up with a brittle environment that is hard to evolve. The developer who reaches for skills first and escalates to hooks only when necessary ends up with a flexible environment whose enforcement layer is concentrated where it actually earns its place.

第三条范式是"hooks 作为最后手段"原则。当开发者面临"用 hook 还是用 skill 来强制一条规约"的选择时,正确的默认是先 skill,只有当 skill 不够用时才升级到 hook。Skill 更软、更灵活、更易维护、也更像对话;hook 更硬、更刚性、出错时也更难排查。"模型只要被提醒一下,就能稳定地遵守"的规约,应当住在 skill 里;"模型真的不能被信赖去遵守、且一旦失败代价极大"的规约,才应当住到 hook 里。这个默认值守住了架构的诚实。先伸手去抓 hook 的开发者,最后会得到一个脆弱、难以演进的环境;先伸手去抓 skill,只在必要时才升级到 hook 的开发者,会得到一个灵活的环境,其强制层恰好集中在它真正值得出场的地方。

The fourth pattern concerns the lifecycle of any specific configuration entry. Every skill, subagent, and hook the developer adds to her environment goes through a predictable lifecycle. It is created in response to a specific friction. It is used heavily for some period. It either earns its place by being used continuously or fades into disuse. Either way, after roughly six months, the developer should review the entry and ask whether it is still pulling its weight. Entries that have not been used in months should be removed. Entries that have been used heavily but slightly off-fit should be refined. Entries that have been used as a starting point for several variants should perhaps be split into more specific siblings. This kind of pruning maintenance is what keeps an environment from accumulating dead weight, and the developers who treat their `.claude` directories as living

第四条范式是关于"任意一条配置项的生命周期"的。开发者每加进环境里一个 skill、一个 subagent、一个 hook,都会走一条可预测的生命周期:因某个具体摩擦而被创建;有一段时间被频繁使用;之后要么靠持续被使用而"赢得自己的位置",要么慢慢淡出。不管哪种,大概 6 个月后,开发者都应当回头 review 一下这条项,问问"它还在撑着场面吗"。几个月没被用过的项,删掉;被频繁用但用得有点别扭的,精修;已经被当作多个变体起点的,考虑拆成几个更专门的兄弟项。这种"剪枝式"的维护,正是让一个环境不至于积累"死重"的关键——而那些把 `.claude` 目录当"活的工件"对待、

projects, gardened regularly, end up with environments that stay sharp for years rather than gradually decaying into clutter.

像对待项目一样、定期打理它的开发者,他们环境会保持锋利好些年,而不是慢慢腐烂成一堆乱糟糟的东西。

The fifth pattern, and the last to name explicitly before the case studies, is the small-and-good principle. A small environment of fifteen well-designed components beats a large environment of fifty mediocre ones, almost always. The temptation to build large is real, especially after the first few components produce visible benefits. The discipline is to resist building large for its own sake, and to add components only when their need is concrete and undeniable. Many of the most effective environments in practice contain fewer than thirty components total. Their effectiveness comes from how well-tuned the components are, not from their count. The reader who internalizes the small-and-good principle will produce environments that work beautifully and remain maintainable. The reader who does not will produce environments that impress at first glance and frustrate at the first attempt to evolve them six months later.

第五条范式,也是进入案例前最后要明说的,叫"小而精"原则。一个 15 个组件、个个设计得好的小型环境,几乎在所有情形下都胜过 50 个组件、但个个平庸的大型环境。诱惑是真实存在的——尤其是前几个组件带来了可见收益之后,人会很容易飘起来。要守的纪律是:不为"大"而大,只在需求具体且不可否认时才加新组件。实践中很多最有效的环境,总数都不到 30 个。这些环境之所以有效,靠的是每个组件都被调得有多好,而不是组件的数量。把"小而精"原则真正吃进去的读者,会造出用起来漂亮、维护起来也轻的环境;吃不进去的,会造出"乍一看令人印象深刻、但六个月后一要演进就让人抓狂"的环境。

A final architectural observation, before the case studies begin, concerns the relationship between the three-pillar architecture and the deeper craft of software engineering. Engineers who have worked on production systems for years recognize the architecture in this book as familiar. Skills resemble the configuration files that have always shaped good engineering systems. Subagents resemble the worker pools and microservice patterns that decompose monolithic systems into focused specialists. Hooks resemble the middleware, the policy enforcement points, and the observability layers that wrap deterministic guarantees around probabilistic services. The three-pillar architecture is not, in its bones, a new invention. It is the application of well-established software-architecture patterns to the specific problem of working with an AI assistant. This continuity is reassuring, because it means that the discipline this book asks the reader to develop is the same discipline that produces good software in general. A developer who builds her Claude Code environment well becomes a slightly better software engineer along the way, because the patterns she practices on her environment are the same patterns she should practice on her production systems. The book's teaching, in this sense, transfers beyond the tool.

进入案例之前,最后一条架构观察,是关于"三柱架构"和"软件工程这门更深的手艺"之间关系的。在生产系统上工作过多年的工程师,会在本书这套架构里认出自己的老熟人。Skills 像那些一直以来"塑造优秀工程系统"的配置文件;subagents 像把单体系统拆成专门化"小而专"的 worker pool 和微服务范式;hooks 像那些"把确定性保证包在概率性服务外头"的中间件、策略执行点和可观测性层。三柱架构骨子里并不是什么新发明;它是把已经成熟的软件架构范式,应用到了"和 AI 助手协作"这个具体问题上。这种连续性让人安心——它意味着本书让读者培养的纪律,和"做好的软件"这件事本身所要求的纪律,其实是同一种。开发者把 Claude Code 环境搭好,顺带会成为稍微好一点的软件工程师,因为她在环境上练的这些范式,正是她应该在生产系统上也练的那些。从这个意义上,本书所教的东西,溢出了工具本身。

CHAPTER TWENTY · 第二十章

Case Study: Building a Full-Stack Development Environment

案例研究:搭建一个全栈开发环境

"The proof of the architecture is in the building. Until something stands, the design is only an opinion."

— a structural engineer, anonymous

"架构是否成立,要在盖起来之后才算数。在有什么东西真正立在那儿之前,设计只是意见。"

—— 一位结构工程师(匿名)

This chapter is a build. The reader will follow, end to end, the construction of a complete Claude Code environment for a single specific kind of work: full-stack web development in TypeScript, with React on the front end and Node with PostgreSQL on the back end. The reader does not need to be a TypeScript developer for the chapter to be useful. The point of the case study is the shape of the construction, not the specific technologies. A reader who works in a different stack can read the chapter as a template and substitute her own languages, her own frameworks, her own conventions. The skeleton is what matters. The flesh is illustrative.

这一章就是一场“搭”。读者会从头到尾跟着,把一份完整的 Claude Code 环境搭出来——针对一种非常具体的工作:TypeScript 全栈 Web 开发,前端 React,后端 Node + PostgreSQL。读者本人并不需要是 TypeScript 开发者,这一章才有价值。这个案例研究的重点,是“搭建的形状”,而不是具体技术栈。读者在用别的技术栈,完全可以把这一章当作一个模板来读,把语言、框架、规约换成自己的。骨架才是重点;血肉只是示例。

The developer in this case study is a senior engineer at a small product company. She works in a single repository that contains a Next.js front end, a Node API server, a PostgreSQL database with Prisma as the ORM, a test suite written in Vitest, and deployment configuration in Docker and Kubernetes. The team has agreed conventions for how new features are scaffolded, how tests are written, how database migrations are produced, how PR descriptions are written, and how releases are tagged. The conventions are presently maintained in a wiki that nobody reads after their first week on the team. The developer wants to encode the conventions into her Claude Code environment so that the environment enforces and applies them automatically, freeing her from having to remember them and freeing her teammates from having to be reminded of them in every code review.

案例里的这位开发者,是某小型产品公司的一位资深工程师。她在单一仓库里工作:Next.js 前端、Node API 服务、用 Prisma 作为 ORM 的 PostgreSQL 数据库、用 Vitest 写的测试套件、Docker + Kubernetes 的部署配置。团队已经就一些规约达成了一致:新 feature 怎么搭脚手架、测试怎么写、数据库 migration 怎么生成、PR description 怎么写、release 怎么打 tag。这些规约目前被维护在一个 wiki 里——而入职第一周之后,没人再去看那个 wiki。这位开发者希望把规约编码进她的 Claude Code 环境,让环境自己去强制、自己去应用,免得她每次都要去记、也免得队友每次在 code review 里被反复提醒。

The build will produce, in the project's `.claude` directory, a configuration that includes eight skills, four subagents, and six hooks. By the end of the chapter, the directory will be committed to the repository, every developer who clones the project will inherit the environment

这次搭建会在项目里的 `.claude` 目录下,产出一份包含 8 个 skills、4 个 subagents、6 个 hooks 的配置。等这一章结束时,这个目录会被提交进仓库;之后凡是 clone 这个项目的开发者,都会自动继承这一环境——

automatically, and the daily experience of working in the project will be substantially different from what it was before the build began.

在这个项目里"日常工作的体验",会和搭建开始之前有明显不同。

Begin with the directory structure. The `.claude` directory at the project root is organized as follows.

先从目录结构开始。项目根目录下的 `.claude` 长这样:

```
.claude/
├── settings.json
├── skills/
│   ├── component-scaffold/
│   ├── api-route-scaffold/
│   ├── prisma-migration/
│   ├── pr-description/
│   ├── commit-message/
│   ├── test-conventions/
│   ├── api-doc/
│   └── error-format/
├── agents/
│   ├── security-auditor.md
│   ├── performance-reviewer.md
│   ├── test-writer.md
│   └── migration-planner.md
└── hooks/
    ├── block-dangerous.sh
    ├── protect-files.sh
    ├── format-on-save.sh
    └── tests-must-pass.sh
```

Each of the components has a specific job in the larger system. The skills encode the project's knowledge. The subagents provide specialized roles for the major review and authoring tasks. The hooks enforce the deterministic guarantees. Together, the configuration makes a substantial development workflow possible to perform in a single Claude Code session, with the environment carrying most of the load that the developer used to carry herself.

每一个组件,都在这套大系统里承担着特定职责。Skills 编码的是项目里的知识;subagents 为"主要的 review 与撰写任务"提供专门化角色;hooks 强制的是确定性保证。三者合起来,使得"在一次 Claude Code 会话里完成一份相当分量的开发工作流"成为可能——原本由开发者自己扛的那大部分负担,现在由环境接管了。

Begin building with the skills, because skills are the foundation. The first skill is `component-scaffold`, which generates a new React component in the project's conventional layout. A React component in this codebase is a folder containing the component file, a test file, a Storybook story, and an index re-export. The folder convention is that each component lives at `src/components/[ComponentName]/`, with PascalCase naming throughout. The skill captures this convention and the templates for each generated file.

先从 skills 搭起——skills 是地基。第一个 skill 是 `component-scaffold` ,作用是按项目的规约布局生成一个新的 React 组件。这个代码库里,一个 React 组件是一个目录,里面装:组件文件本身、一份测试文件、一份 Storybook story、以及一个 index 的再导出。规约上的目录约定是:每个组件住在 `src/components/[ComponentName]/` 下,全程 PascalCase 命名。这个 skill 把这条规约和每个被生成文件的模板一起,都收进了它的定义里。

← 前一页

后一页 →

The second skill is `api-route-scaffold` , which generates a new API route in the Next.js conventional layout, with the route handler, the validation schema, the test file, and the OpenAPI documentation fragment. The skill captures the team's convention that every route has explicit validation using Zod, every route has a test that exercises the happy path and at least two error cases, and every route has a corresponding OpenAPI fragment that lives alongside the route file.

第二个 skill 是 `api-route-scaffold` ,作用是按 Next.js 的规约布局生成一条新的 API 路由:包含路由 handler、validation schema、测试文件、以及 OpenAPI 文档片段。这个 skill 把团队规约也一起收进去了:每条路由必须用 Zod 做显式校验;每条路由的测试必须覆盖 happy path 加至少两种错误情形;每条路由必须有一份与之同址存放的 OpenAPI 片段。

The Component Scaffold Skill · 组件脚手架 skill

```
---
name: component-scaffold
description: Scaffolds a new React component in the project's conventional layout, complete with test file, Storybook
allowed-tools: Read, Write, Glob
---
# Component Scaffold

When asked to scaffold a component, follow this procedure.

1. Get the component name from the user. Convert to PascalCase if needed.
2. Create the folder at `src/components/[ComponentName]/`.
3. Create the four files using the templates in `assets/`:
  - `[ComponentName]/index.ts`, re-exports the component as a named export
  - `[ComponentName]/[ComponentName].tsx`, the component itself
  - `[ComponentName]/[ComponentName].test.tsx`, Vitest test file
  - `[ComponentName]/[ComponentName].stories.tsx`, Storybook story
4. Substitute the placeholder `__COMPONENT_NAME__` with the actual name in each file.
5. Confirm by listing the four files created.

Use functional components with hooks. Use TypeScript strict mode. Define props as a named interface above the compon
```

The API Route Scaffold Skill · API 路由脚手架 skill

```
---
name: api-route-scaffold
description: Scaffolds a new API route in the Next.js conventional layout, complete with handler, Zod validation sch
allowed-tools: Read, Write, Glob
---
# API Route Scaffold
```

When asked to scaffold an API route, follow this procedure. (正文接下页)

注:上面这段 YAML 是该 skill 的核心定义。name 字段是 skill 的标识; description 字段是触发语言; allowed-tools 限制 skill 激活时可用的工具集;body 部分(--- 之后)则是 Claude 在执行时遵循的步骤与规约。

注:上面这段 YAML 是该 skill 的核心定义。name 字段是 skill 的标识; description 字段是触发语言; allowed-tools 限制 skill 激活时可用的工具集;body 部分(--- 之后)则是 Claude 在执行时遵循的步骤与规约。

← 前一页

后一页 →

The Migration Skill · Migration skill

The third skill is `prisma-migration`, which produces a Prisma migration file from a description of the schema change, with the up migration, the down migration, and a note about whether the change is backward-compatible.

第三个 skill 是 `prisma-migration`,作用是:给定一段对 schema 变更的描述,产出一份 Prisma migration 文件——包含 up migration、down migration、以及一条关于"这次变更是否向后兼容"的说明。

API Route Scaffold(续) · API 路由脚手架(续)

When asked to scaffold an API route, follow this procedure.

1. Get the route path and HTTP method from the user.
2. Create the route handler at ``app/api/[path]/route.ts``.
3. Create the validation schema at ``app/api/[path]/schema.ts``. All input must be validated with Zod before being processed.
4. Create the test file at ``app/api/[path]/route.test.ts``. Cover happy path, validation failure, and one business-logic error.
5. Create the OpenAPI fragment at ``app/api/[path]/openapi.yaml``. Document the request shape, the response shape, and the error shape.
6. Wire the route into the API surface following the project's convention.

Every route must validate its input with Zod. Every route must produce errors in the project's standard error format.

Prisma Migration Skill · Prisma Migration skill

```
---
name: prisma-migration
description: Produces a Prisma migration from a described schema change, with up and down migrations and a backward-compatible note.
allowed-tools: Read, Write, Bash
---
# Prisma Migration
```

When asked to create a migration, follow this procedure.

1. Read the current schema at ``prisma/schema.prisma``.
2. Apply the change to the schema. Be conservative: prefer additive changes (new columns, new tables) over destructive changes.
3. Generate the migration file using ``npx prisma migrate dev --create-only --name [descriptive_name]``.
4. Read the generated migration. Verify it does what was intended.
5. If the migration is destructive, write a note at the top of the migration file explaining what is being changed and why.

6. Report:
- What changed

注:以上是 `prisma-migration` skill 的核心定义,body 的第 6 步"报告什么变了"在下一页继续展开。

注:以上是 `prisma-migration` skill 的核心定义,body 的第 6 步"报告什么变了"在下一页继续展开。

← 前一页

后一页 →

The fourth skill is `pr-description`, which produces a pull request description in the team's standard format. This is a near-copy of the personal `pr-description` skill from Chapter Nine, adjusted for the team's specific conventions.

第四个 skill 是 `pr-description`,作用是按团队的标准格式生成一份 pull request 描述。这个 skill 和第九章里的"个人版 `pr-description` skill"几乎是同一份,只是按本团队的具体规约做了几处微调。

Prisma Migration(续) · Prisma Migration(续)

- Whether the change is backward-compatible (can old code run against the new schema)
- Whether a data migration is needed in addition to the schema migration
- Recommended deploy order (migration before code, or code before migration)

Do not run ``prisma migrate deploy`` or apply the migration. The user reviews and applies it manually.

PR Description Skill · PR Description skill

```
---
name: pr-description
description: Drafts a pull request description in the team's standard format. Use whenever the user asks for a PR de
allowed-tools: Bash, Read, Grep
---
# PR Description

When asked for a PR description, follow this procedure.

1. Identify the diff. Run `git log` and `git diff` against the merge base on main.
2. Read the diff in full.
3. Produce the description in this exact format:

## Summary
A single sentence in the imperative mood describing the change.

## What changed
A bulleted list of concrete changes, one bullet per logical unit.

## Schema changes
If the diff includes any Prisma migrations, summarize the schema changes here. State whether they are backward-compa

## Testing
Describe how the change was tested. List any new test files created.
```



```
## Risk
One sentence categorizing risk as low, medium, or high with a one-line reason.
```

注:上面这段是 `pr-description` skill 的核心定义,body 中"Deploy notes"小节将在下一页继续。

注:上面这段是 `pr-description` skill 的核心定义,body 中"Deploy notes"小节将在下一页继续。

← 前一页

后一页 →

The fifth skill is `commit-message`, identical to the Chapter Five version but installed at the project level rather than the user level so that every developer in the team uses the same conventions.

第五个 skill 是 `commit-message`,和第五章那一份完全一致,只是装在 `project` 级别而不是 `user` 级别——这样团队里的每一位开发者都会使用同一套规约。

The sixth skill is `test-conventions`, which encodes the project's specific test conventions: Vitest, the location pattern for test files, the use of MSW for HTTP mocking, the convention of one `describe` block per function and one `it` block per scenario. The skill is loaded by the `test-writer` subagent when it begins working, so that the subagent automatically applies the project's conventions without needing to be briefed every time.

第六个 skill 是 `test-conventions`,作用是把本项目特有的测试规约编码进去:Vitest、测试文件的存放位置模式、用 MSW 做 HTTP 模拟、每个函数对应一个 `describe` 块、每个场景对应一个 `it` 块等。当 `test-writer` subagent 一开始工作,就会加载这个 skill ——这样 subagent 就能自动套用本项目规约,不必每次被人提醒。

PR Description(续) · PR Description(续)

```
## Deploy notes
Any non-default deployment considerations, such as environment variable changes, manual data migrations, or required

Do not invent changes that do not appear in the diff.
```

Test Conventions Skill · Test Conventions skill

```
---
name: test-conventions
description: Encodes this project's test conventions, including framework choice, file layout, mocking patterns, and
allowed-tools: Read
---
# Test Conventions

This project uses Vitest. Test files are colocated with the file under test, with a `.test.ts` or `.test.tsx` suffix

## Structure
- One `describe` block per public function or component.
- One `it` block per scenario. Every public function should have at minimum: happy path, one boundary, one error.
```

– Use `beforeEach` for setup that resets state. Avoid `beforeAll`.

Mocking

- Use Vitest's `vi.mock` for module mocks.
- Use MSW (Mock Service Worker) for HTTP mocks. Set up handlers in the test file's `beforeAll`.
- Never make real network calls in tests.
- Never write to the real file system in tests. Use `memfs` if file operations are needed.

注:上面这段是 `test-conventions` skill 的核心定义,剩余的"## Database"、"## Async"、"## Naming"三个小节将在下一页继续。

注:上面这段是 `test-conventions` skill 的核心定义,剩余的"## Database"、"## Async"、"## Naming"三个小节将在下一页继续。

← 前一页

后一页 →

The seventh skill is `api-doc`, which produces an OpenAPI documentation fragment for a given route. It is similar to the `api-route-scaffold` skill in that it produces OpenAPI output, but its job is to update existing documentation rather than to create scaffolding for new routes.

第七个 skill 是 `api-doc`,作用是:给定一条路由,产出它对应的 OpenAPI 文档片段。它和 `api-route-scaffold` 相似——两者都产出 OpenAPI 文档——但它的工作是"更新已存在的文档",而不是为新路由搭脚手架。

The eighth skill is `error-format`, which encodes the team's standard error format. The format includes a numeric code, a human-readable message, and an optional details object. The skill is short but loaded by other skills and subagents whenever they need to produce or parse errors.

第八个 skill 是 `error-format`,作用是:把团队标准的错误格式编码进去。格式包含一个数字 code、一条人类可读的消息、以及一个可选的 details 对象。这个 skill 本身不长,但其他 skill 和 subagent 一旦需要产出或解析错误,就会去加载它。

With the eight skills in place, the project has its conventions encoded as a small library that any session in the project can draw on automatically. Move next to the subagents, four of which complete the specialist team.

8 个 skills 各就各位之后,项目的规约就被编码成了一座小巧的库——项目中任何一个 session 都可以自动取用。接下去是 subagents,4 个 subagent 凑齐这支"专家小队"。

The first subagent is `security-auditor`, identical in shape to the version from Chapter Eleven. The second is `performance-reviewer`, also identical to its Chapter Thirteen counterpart. Both are general-purpose specialists that benefit from the project's skills when they activate.

第一个 subagent 是 `security-auditor`,结构上和第十一章的版本完全一致。第二个是 `performance-reviewer`,也和第十三章的对应物完全一致。这两个都是通用型的专家,激活时还能享受到项目里 skills 的加持。

The Test Writer Subagent · Test Writer subagent

The third subagent is `test-writer`, customized slightly for this project. The customization is in the description, which now mentions Vitest and the project's specific test conventions, and in the system prompt, which directs the subagent to load the `test-conventions` skill at the start of its work.

第三个 subagent 是 `test-writer`, 针对本项目做了轻微定制: 定制落在两处——description 中点名了 Vitest 和本项目特有的测试规约; system prompt 中指示这个 subagent 一开工就先把 `test-conventions` skill 加载起来。

Test Conventions(续) · Test Conventions(续)

```
## Database
- Tests that touch the database use a transaction that is rolled back after the test.
- Never use the production or staging database for tests.

## Async
- Always await async operations explicitly. Do not rely on returned promises being awaited by the test runner.
- Use `vi.useFakeTimers()` for tests that depend on time.

## Naming
- Test files: `[FileName].test.ts` or `[ComponentName].test.tsx`.
- Test descriptions: imperative, e.g. "creates a user when given valid input," not "should create a user."
```

The fourth subagent is `migration-planner`, used when the developer is contemplating a substantial migration such as a Next.js major-version upgrade or a switch from Prisma to a different ORM. The subagent is identical in shape to its Chapter Thirteen version.

第四个 subagent 是 `migration-planner`, 用于“开发者正在认真考虑一次大型 migration”的情形——比如 Next.js 的主版本升级, 或者把 Prisma 换成别的 ORM。它的结构和第十三章的版本完全一致。

With four subagents installed, the project has access to specialized parallel review and authoring capabilities for security, performance, testing, and migration planning. The orchestrator pattern from Chapter Twelve

4 个 subagent 装好之后, 这个项目就有了“安全、性能、测试、migration 规划”这四个维度上的专门化并行 review 与撰写能力。第十二章的 orchestrator 范式——

Test Writer Subagent · Test Writer subagent 定义

```
---
name: test-writer
description: Writes Vitest test files for a given function or module, following this project's test conventions. Use
tools: Read, Write, Bash, Glob
model: sonnet
---

You are a senior test engineer. Your job is to write tests that catch real bugs, not tests that only exercise the ha
```

Your method:

1. Activate the project's test-conventions skill at the start of your work.
2. Read the file under test and any closely related files needed to understand its behavior.
3. Identify the public surface of the file: functions, methods, classes that other code calls.
4. For each item on the public surface, write tests covering at minimum:
 - The happy path
 - At least one boundary condition
 - At least one error or invalid-input case
5. Mock external dependencies using the project's mocking patterns (Vitest `vi.mock` for modules, MSW for HTTP).
6. Place test files alongside the code under test, following the project's naming convention.

What you do not do:

- You do not modify the code under test, even if you discover bugs while writing tests. Note the bug as a comment in
- You do not write tests that only call the function and assert that it does not throw.
- You do not mock the function under test. You mock its dependencies.
- You do not produce excess of nearly-identical tests. Three meaningful tests beat ten redundant ones.

End with a list of the test files you wrote and any bugs or concerns you noticed in the code under test.

注:以上是 `test-writer` subagent 的完整定义。`tools` 字段限定它可用的工具集(本案例中拿不到 `WebFetch`); `model: sonnet` 指明它跑在 Sonnet 模型上;System prompt 用了"方法步骤 + 显式禁止项"的双层结构,以避免它写出"只验证函数不抛异常"那种空壳测试。

注:以上是 `test-writer` subagent 的完整定义。`tools` 字段限定它可用的工具集(本案例中拿不到 `WebFetch`); `model: sonnet` 指明它跑在 Sonnet 模型上;System prompt 用了"方法步骤 + 显式禁止项"的双层结构,以避免它写出"只验证函数不抛异常"那种空壳测试。

← 前一页

后一页 →

activates naturally when the developer asks for broad reviews, and the four subagents work in parallel to produce a multidimensional report.

在开发者提出"宽口径 review"请求时就会被自然触发,4 个 subagent 并行工作,产出一份多维度的报告。

Move next to the hooks. Six hooks complete the configuration, and they live in the `.claude/hooks` directory as separate scripts referenced from the `settings.json` file. The `settings.json` itself looks like this.

接下去是 hooks。6 个 hooks 收尾整个配置;它们住在 `.claude/hooks` 目录下,作为独立的脚本被 `settings.json` 引用。
`settings.json` 本身长这样:

Settings.json · settings.json(节选,接续下页)

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
```

```

        "type": "command",
        "command": "echo '## Branch'; git branch --show-current; echo '## Status'; git status --short"
    }
}
],
"PreToolUse": [
{
    "matcher": "Bash",
    "hooks": [
        {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/block-dangerous.sh"
        }
    ]
},
{
    "matcher": "Edit|Write|MultiEdit",
    "hooks": [
        {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/protect-files.sh"
        }
    ]
}
],
"PostToolUse": [
{
    "matcher": "Edit|Write|MultiEdit",
    "hooks": [
        {
            "type": "command",

```

注:这段 JSON 是 `settings.json` 的开头。完整结构由四个事件构成: `SessionStart` (打 git 状态)、`PreToolUse` (两道门: `block-dangerous.sh` 给 Bash、`protect-files.sh` 给文件编辑)、`PostToolUse` (运行格式化与测试)、`Stop` (检查未提交变更)。余下内容在下一页继续。

注:这段 JSON 是 `settings.json` 的开头。完整结构由四个事件构成: `SessionStart` (打 git 状态)、`PreToolUse` (两道门: `block-dangerous.sh` 给 Bash、`protect-files.sh` 给文件编辑)、`PostToolUse` (运行格式化与测试)、`Stop` (检查未提交变更)。余下内容在下一页继续。

← 前一页

后一页 →

The `block-dangerous.sh` and `protect-files.sh` scripts are the same security gates from Chapter Seventeen, slightly expanded for this project to add the production deployment paths and the `.env.production` file as protected. The `format-on-save.sh` script runs Prettier and ESLint on every file that Claude edits or writes. The `tests-must-pass.sh` script runs the test suite scoped to the changed file's area, and if any tests fail, returns a JSON decision-block that surfaces the failure to Claude with a directive to fix the failing tests before continuing. Together, the four `PreToolUse` and `PostToolUse` hooks form a tight enforcement pipeline that runs on every action Claude takes inside the project.

`block-dangerous.sh` 和 `protect-files.sh` 这两道门和第十七章那两道一样——只是为本项目做了点小扩展:把"生产部署路径"和 `.env.production` 也加进了受保护名单。`format-on-save.sh` 会在 Claude 每次编辑或写文件时跑一遍 Prettier 和 ESLint。`tests-must-pass.sh` 则会跑"限定在被改文件所属区域"的测试套件;一旦有失败,会回一份 JSON decision-block,把失败暴露给 Claude,要求它在继续之前把失败的测试修好。四道 PreToolUse + PostToolUse hook 合起来,组成了一个紧贴的强制流水线——Claude 在项目里每动一下,都会被它"过一遍"。

The `Stop` hook is a final guard against ending a session with uncommitted changes. The hook checks the `git status` at session end, and if any changes are present, it returns a JSON decision-block that prevents the session from ending and surfaces a clear instruction to commit or stash before continuing. This is a small but real piece of workflow enforcement; it makes it impossible to walk away from the terminal with work in an inconsistent state.

`Stop` hook 是最后一道门,防止你"带着未提交的变更就退出 session"。这个 hook 会在 session 结束时检查 `git status`;一旦发现有任何未提交的变更,就回一份 JSON decision-block,拦住 session 退出,并把一条清晰的指令推到面前——"先 commit 或 stash 才能继续"。这是一条小但真实的"工作流强制":它让人不可能从终端前走开时,工作还处在不一致的状态里。

Settings.json(续) · settings.json(续)

```
        "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/format-on-save.sh"
      },
      {
        "type": "command",
        "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/tests-must-pass.sh"
      }
    ]
  },
  "Stop": [
    {
      "hooks": [
        {
          "type": "command",
          "command": "if [ \"$(git status --porcelain | wc -l)\" -gt 0 ]; then echo '{\"decision\":\"block\",\"rea"
        }
      ]
    }
  ]
}
```

注:以上是 `settings.json` 的下半段。PostToolUse 在文件被改/写后跑 `format-on-save.sh` + `tests-must-pass.sh`; Stop 在 session 结束那一刻检查 `git status --porcelain` ——只要行数大于 0,就把"未提交变更"这条理由回给 Claude,强制它继续直到退出条件满足。

注:以上是 `settings.json` 的下半段。PostToolUse 在文件被改/写后跑 `format-on-save.sh` + `tests-must-pass.sh`; Stop 在 session 结束那一刻检查 `git status --porcelain` ——只要行数大于 0,就把"未提交变更"这条理由回给 Claude,强制它继续直到退出条件满足。

The configuration is now complete. Eight skills, four subagents, six hooks. The `.claude` directory is committed to the repository, and from the next session forward, every developer who works in this project inherits the entire environment. The work that previously required the developer to remember the conventions, apply them manually, and verify them after the fact, now runs automatically. The conventions are encoded in skills. The specialist work is dispatched to subagents. The boundaries are enforced by hooks.

配置到此完成。8 个 skills、4 个 subagents、6 个 hooks。`.claude` 目录被提交进仓库;从下个 session 开始,在本项目里工作的每一位开发者,都会继承这一整套环境。原本要由开发者自己"记规约、手工应用、再事后核对"的那一大堆工作,现在自动就跑了。规约编码在 skills 里;专家工作派给 subagents;边界由 hooks 守着。

A Day in the Environment · 在这个环境里的一天

A walk through a representative day in the project will make the difference visible. The developer opens Claude Code in the morning. The `SessionStart` hook fires and shows her the current branch and any uncommitted changes from the previous day, which there are none of, because the `Stop` hook from the previous session would have prevented her from leaving the terminal with uncommitted work. She decides to add a new feature: a user profile endpoint with avatar upload support. She types into Claude Code, in plain language, scaffold an API route at `/api/users/profile` that accepts PUT requests with a multipart body containing an avatar file.

走一遍这个项目里"代表性的一天",可以让这种差别变得可见。开发者早晨打开 Claude Code。`SessionStart` hook 触发,把当前 branch 和前一天留下的未提交变更显示给她——这次什么也没有,因为上一个 session 的 `Stop` hook 早就不允许她"带着未提交的工作走开"。她决定加一个新 feature: 一个支持上传头像的 user profile 接口。她用大白话在 Claude Code 里敲:给我搭一个 `/api/users/profile` 路由,接受带 multipart body 的 PUT,body 里装一个头像文件。

The `api-route-scaffold` skill fires automatically because the request matches its description. Claude creates the route handler, the Zod schema, the test file with three tests, and the OpenAPI fragment. The `format-on-save` hook runs Prettier on each new file as it is written. The `tests-must-pass` hook runs the project's tests after each write; the new tests pass, because the scaffold includes them complete, and the hook is silent.

`api-route-scaffold` skill 自动触发——因为请求匹配上了它的 description。Claude 创出 route handler、Zod schema、含三个测试的测试文件、还有 OpenAPI 片段。`format-on-save` hook 在每个新文件落地时跑一遍 Prettier。`tests-must-pass` hook 在每次写完后跑一遍项目测试;新测试全部通过(因为脚手架把它们都生成了完整),于是 hook 静默。

The developer then asks for a commit. The `commit-message` skill fires. Claude reads the staged changes, produces a conventional commit message in the project's format, and presents it. The developer approves it. The commit happens. The `format-on-save` hook runs again on each file, finds nothing to change, and exits silently.

开发者接着要一次 commit。`commit-message` skill 触发。Claude 读 staged changes,按项目格式生成一条 conventional commit 消息,呈到面前。开发者点了 ok,commit 完成。`format-on-save` hook 又跑一遍每个文件——没东西要改——于是又静默退出。

She then asks for a PR description. The `pr-description` skill fires. Claude reads the branch's diff against main, produces a PR description in the team's exact format with all five sections including the deploy notes section that flags the new endpoint as needing the file-upload size limit increased in the production environment, and presents it. She copies it into the GitHub PR.

接着要一份 PR 描述。`pr-description` skill 触发。Claude 读 branch 相对 main 的 diff,按团队精确的格式生成 PR 描述——五个小节一个不少——其中"Deploy notes"小节特别指出:新接口上线需要在生产环境调高文件上传大小限制。她把它原样粘到 GitHub PR

She wants a thorough review of the change before requesting reviewers. She types, perform a full review of this branch covering security, performance, testing, and architecture. The orchestrator pattern recognizes the multidimensional shape of the request and dispatches four subagents in parallel: security-auditor , performance-reviewer , the test-writer (in review mode), and an architecture-reviewer . Each runs in its own context. Each loads the project skills it needs. Two minutes later, four reports return. The unified review notes that the file upload code does not validate the file size on the server side (security flag), that the avatar processing happens synchronously in the request handler when it should be queued (performance flag), and that the test coverage is solid. She addresses the security and performance flags before requesting reviewers.

在叫 review 之前,她想对这次变更做一次彻底的审查。她敲:对这个 branch 做一次完整 review,覆盖安全、性能、测试、架构。Orchestrator 范式识别出请求的"多维度"形状,并行派出 4 个 subagent: security-auditor 、 performance-reviewer 、 test-writer (review 模式)、以及 architecture-reviewer 。每个 subagent 跑在各自的上下文里,各自加载项目里它需要的那些 skill。两分钟后,4 份报告回来。综合 review 指出:文件上传那段代码在服务端没校验文件大小(安全 flag);avatar 处理在 request handler 里同步发生,本该走队列(性能 flag);测试覆盖是扎实的。她在叫 reviewer 之前,先把安全和性能两条 flag 处理掉了。

The whole interaction has taken her twenty-five minutes. Without the environment, the same work would have taken closer to two hours, and several of the issues the review caught would likely have been merged in and discovered later. The environment did the conventions work. The environment did the review work. The environment enforced the boundaries. She did the design and judgment work, which is what she was hired for in the first place.

整个交互花了 25 分钟。没有环境的话,同样的活会接近两小时,而且 review 抓出来的那几条问题很可能就这么被合进去,之后才被发现。环境做了规约的工作;环境做了 review 的工作;环境守住了边界。她做的,是"设计和判断"那部分工作——这本来就是公司雇她来做的事。

A reflection on what the case study demonstrates. The configuration above is not large. The .claude directory at the end of the build is roughly three thousand lines of configuration spread across twenty files. A single developer can build it in a long afternoon. Once built, it persists. It travels with the project to every developer who clones the repository. It saves substantial time on every session in the project, indefinitely. The investment-to-benefit ratio is exceptional, and the case study is meant to convince the reader that her own project has a similar configuration latent inside it, waiting to be built.

回头想想这个案例证明了什么。上面这份配置并不大。搭完之后,这个 .claude 目录大致是 3000 行配置,分布在 20 个文件里。单个开发者花一个长一点的下午就能搭起来。一旦搭好,它就持久存在——它会随项目走到每一位 clone 仓库的开发者那里。它会在项目里的每一个 session 上持续省下大把时间,没有止境。投入产出比出奇地高;这个案例的目的是说服读者:你自己的项目里,也有一个类似配置在潜伏着,等着被搭出来。

A second walkthrough of a different kind of session is worth including, because the morning scenario above showed the environment helping with a relatively well-defined feature build. A different kind of session, the bug investigation, exercises different parts of the environment and is worth seeing in concrete form.

第二种 session 演示也值得放进案例里。上面那个"早晨场景"展示的是环境帮上"形态良好、定义清晰的 feature 搭建";另一种 session——bug 调查——会让环境里不同的部分被锻炼到,也值得以具体形态呈现一遍。

A Bug Investigation · 一次 bug 调查

The developer notices that errors from the avatar upload endpoint have been spiking in the production monitoring dashboard. She opens Claude Code in the project. The `SessionStart` hook fires and shows her the current branch (a feature branch she had been working on yesterday) and any uncommitted changes (none, because the `Stop` hook from the previous session prevented her from leaving with dirty state). She switches to main and pulls the latest. She types into Claude Code, recent errors from `/api/users/profile` suggest something is failing on large file uploads; investigate.

开发者注意到生产监控面板上,avatar upload 接口的错误数在飙升。她在项目里打开 Claude Code。`SessionStart` hook 触发,把当前 branch(她昨天在写的一个 feature branch)和未提交变更(没有——上一个 session 的 `Stop` hook 早就不允许她"带着脏状态走")显示给她。她切到 main,把最新代码拉下来。她在 Claude Code 里敲:最近 `/api/users/profile` 的错误说明,大文件上传那条路径上有什么东西在挂;请查。

The orchestrator pattern recognizes the investigative shape of the request, but unlike the morning's feature work, no existing subagent perfectly matches investigation. The main session handles the work directly, reading the route handler, reading the recent commits to the file, reading the error logs the developer points it at. It identifies a likely cause: the Zod schema does not enforce a maximum file size, and the underlying middleware that should have caught oversized uploads has a configuration drift between staging and production. The main session proposes a fix: tighten the schema to include explicit size validation, and adjust the middleware configuration. The developer agrees with the diagnosis.

Orchestrator 范式识别出这是"调查型"的请求——但和早上的"feature 搭建"不同,这次没有任何一个现成的 subagent 能完美匹配"调查"这件事。主会话直接接手:读 route handler、读这份文件最近的 commits、读开发者指过来的错误日志。它定位到一个可能的原因:Zod schema 没有强制最大文件大小,而本来应该兜住"过大上传"的那道中间件,在 staging 和 production 之间有配置漂移。主会话提出修复方案:把 schema 收紧,显式校验大小;再调一下中间件配置。开发者同意这个诊断。

She types, write the fix and add a test for the size validation. The `api-route-scaffold` skill is not the right fit here, because the route already exists. The `test-writer` subagent activates because the request matches its description. The subagent reads the route, reads the existing test file, adds three new test cases for size validation including one boundary, one over-size, and one with the new error format, and ends with a note about the test it wrote. The `format-on-save` hook runs Prettier on the modified test file. The `tests-must-pass` hook runs the tests; they pass. She asks for a commit. The `commit-message` skill produces a message: `fix(users): enforce avatar size limit at the API boundary`. She commits, pushes, opens a PR, asks for a PR description, the `pr-description` skill produces it. The whole investigation takes forty minutes, from noticing the error spike to opening the PR.

她敲:写这个修复,再加一个 size 校验的测试。`api-route-scaffold` skill 不合适——因为这条路由已经存在。`test-writer` subagent 被激活,匹配上了 description。Subagent 读路由、读现有测试文件,补了 3 个 size 校验的新 case:一个边界、一个超限、一个新错误格式;最后附一句对所写测试的说明。`format-on-save` hook 在被改的测试文件上跑一遍 Prettier。`tests-must-pass` hook 跑测试——全过。她要 commit; `commit-message` skill 吐出一条消息: `fix(users): enforce avatar size limit at the`

API boundary。Commit、push、开 PR、要 PR 描述——pr-description skill 吐出来。她从注意到错误飙升,到开 PR,整条链路 40 分钟。

The example shows the environment functioning at a different scale from the morning scenario. No single subagent dominated the work. No single skill was the centerpiece. The environment's value showed up in the small accelerations along the way: the hook that surfaced the branch state at session start, the subagent that wrote the tests in the project's conventions, the formatter and test runner that ran without prompting, the commit-

这个例子说明:环境和"早晨场景"里的运作尺度是不同的。这次没有哪个 subagent 是绝对主角,也没有哪个 skill 占据舞台中央。环境的价值,体现在这一路上无数处"小加速"上:session 一开就把 branch 状态摆出来的 hook、按项目规约写测试的 subagent、不用催就自动跑的格式化和测试、commit 时那

← 前一页

后一页 →

message and PR-description skills that produced consistent outputs at the moments they were needed. The cumulative effect of these small accelerations is what makes the environment substantially better than vanilla Claude Code, and the cumulative effect is most visible in routine sessions like this one rather than in the showcase orchestration of the morning scenario.

刻吐出来、随用随到的 commit 消息和 PR 描述 skill。这些"小加速"加在一起的累积效应,正是这个环境能比 vanilla Claude Code 好出一截的关键——而且这种累积效应,在"日常 session"里反而比"早上的 showcase 编排"更看得见。

A third reflection concerns the team-adoption arc that this kind of environment enables. Once the configuration is committed to the repository, every developer on the team experiences the same environment. New team members inherit it on their first clone. Conventions that previously had to be taught through code review or onboarding documentation are now enforced by the tooling itself. The team's collective output becomes more consistent. The collective time saved compounds across every developer. A team of five developers, each saving ninety minutes a day through their inherited environment, recovers nearly forty hours a week of engineering time. That recovered time can be spent on new features, on technical debt, on the hard problems that the conventions could never have solved on their own. This is the deeper argument for treating the .claude directory as a first-class engineering asset alongside the source code: the leverage compounds across the team, not just within any one developer.

第三条反思,是关于"这类环境所支持的团队采纳曲线"。配置一旦进了仓库,团队里每一位开发者就都处在同一个环境里。新成员一 clone 就自动继承。曾经必须通过 code review 或 onboarding 文档来教的那些规约,现在由工具自己强制。团队整体的产出变得更一致;每一份被省下来的时间,在所有开发者身上一一起复利。一个 5 人的团队,每个人每天靠这个继承来的环境省下 90 分钟,等于一周给团队回收将近 40 小时的工程时间。回收出来的时间,可以投到新 feature、技术债、或规约本身永远解不开的那些难题上。这正是"把 .claude 目录当成和源代码并列的一等工程资产"更深的理由:这种杠杆,是在整个团队层面复利,而不是只复利在某一个开发者身上。

A second reflection concerns the team dimension. The configuration above is committed to the repository, which means that every developer on the team uses the same skills, the same subagents, and the same hooks. When a new developer joins the team, she receives the entire environment automatically when she clones the project for the first time. She does not need to be onboarded into the conventions, because the conventions are enforced by the environment. The wiki that nobody read becomes irrelevant, because the wiki has been replaced by configuration that the tool itself honors. This is how engineering conventions actually take hold in a team: not through documentation, not through training, but through tooling that makes the right thing automatic and the wrong thing impossible. The Claude Code environment is a

particularly powerful version of this pattern, because it operates at the level of the developer's primary AI assistant, which means that almost every action a developer takes inside Claude Code is shaped by the environment.

第二条反思,是关于"团队维度"的。上面这份配置被提交进了仓库,这就意味着团队里每一位开发者都在用同一套 skills、同一套 subagents、同一套 hooks。当一位新开发者加入团队,她第一次 clone 项目时,就把整个环境自动收到了手。她不需要被"onboarding 进规约"——因为规约已经由环境在强制了。那份没人读的 wiki 也就变得无关紧要:它已经被"工具自己会遵守的配置"取代了。这才是工程规约在团队里真正扎根的方式——不是靠文档,不是靠培训,而是靠那种"让对的事自动发生、让错的事无法发生"的工具。Claude Code 环境是这种范式的一个特别有力的版本,因为它作用在"开发者的主要 AI 助手"这一层——也就是说,开发者在 Claude Code 里几乎做的每一件事,都会被环境塑形。

← 前一页

后一页 →

A final reflection on the case study concerns the cost of building it. The afternoon investment to produce the configuration above is real but bounded. A developer who has read this book carefully and who has practiced the patterns on a few personal skills can produce an environment of this size in roughly six to eight hours of focused work. The same developer working on a fresh codebase she has not used Claude Code in before would need closer to twelve hours, because the project conventions have to be written down somewhere before they can be encoded in skills. Either way, the investment is on the order of a single working day, and the return on that investment compounds across every future session. A developer who saves an average of ninety minutes per working day through her environment recovers her initial investment in less than a working week. After that, the savings are pure leverage. This is the practical case for treating the build of the `.claude` directory as a high-priority engineering task rather than as a side hobby. The numbers favor the build, and they favor it dramatically.

关于这个案例,最后一条反思是"搭建成本"。上面这份配置,花一个下午搭出来——投入真实,但有上限。一个认真读完本书、且已经把若干范式在几个人 skill 上练过手了的开发者,大概 6~8 小时专注工作就能搭出这个体量的环境。同一位开发者,在一个她从未用 Claude Code 跑过的全新代码库上,需要接近 12 小时——因为项目规约总得先在某处被写下来,然后才能编码进 skill。两种情形,投入的量级都在"一个工作日"这个量级;而回报会在此后每一个 session 里持续复利。一个开发者,如果通过她的环境每个工作日平均省下 90 分钟,那不到一个工作周就能回本。回本之后,省下来的每一分钟都是纯杠杆。这正是"把搭 `.claude` 目录当作高优先级工程任务、而不是业余消遣"的最朴素理由。数字站在"搭"这一边,而且站得很有力。

The next chapter takes the same architecture and applies it to a very different shape of work. The full-stack developer in this case study had a clearly engineering-shaped job. The next case study features a solo technical founder whose work spans product, marketing, and operations, and whose Claude Code environment looks substantially different from this one even though the underlying architecture is identical. The contrast is instructive, because it shows that the three-pillar framework is not specific to engineering work; it is a general pattern for engineering one's relationship with an AI assistant, and the pattern adapts cleanly to almost any kind of knowledge work. The second case study begins in the chapter ahead.

下一章把同一套架构套到了一种非常不同的工作形状上。本案例里的全栈开发者,做的是"形态明显的工程工作"。下一个案例的主角,是一位独立技术创业者——她的工作横跨产品、营销、运营,她的 Claude Code 环境看上去会和这一份相当不同,但底下那套架构是同一个。这种对比很有教益:它说明三柱框架并不是工程工作特有的——它是"工程化地处理你和 AI 助手关系"的一条通用范式,几乎能干净地适配到任何一种知识工作上。第二个案例,下一章开始。

CHAPTER TWENTY-ONE · 第二十一章

Case Study: The Solo Technical Founder's Claude Code Setup

案例研究:独立技术创业者的 Claude Code 配置

"The same architecture, applied to different work, produces different environments. The architecture is the constant. The work is the variable."

— a software architect, anonymous

"同一套架构,套到不同的工作上,会产生不同的环境。架构是常量;工作才是变量。"

—— 一位软件架构师(匿名)

This chapter is the second case study, and it is deliberately different in shape from the first. The previous chapter built an environment for a single specific kind of engineering work. This chapter builds an environment for a solo founder whose work spans product, engineering, marketing, customer support, and operations, all from a single Claude Code installation. The shift in shape is the point of the chapter. The three-pillar framework was developed in this book using engineering examples, but the framework is not specific to engineering. It is a general pattern for engineering one's relationship with an AI assistant, and the pattern adapts cleanly to almost any kind of knowledge work. By the end of this chapter, the reader will have seen the same architecture produce two very different environments, and she will have the conceptual tools to design a third for her own situation, whatever that situation looks like.

这一章是第二个案例研究,它在形态上故意和第一个拉开了距离。上一章搭的是一个面向"单一特定种类工程工作"的环境;这一章要搭一个独立创业者的环境——她的工作横跨产品、工程、营销、客户支持、运营,所有这些事都从同一份 Claude Code 安装里跑。这种形态上的位移,正是本章的重点。三柱框架在本书里是用工程例子展开的,但它并不专属工程。它是一条"工程化地处理你和 AI 助手关系"的通用范式,几乎能干净地适配到任何一种知识工作上。到本章结束时,读者会看到同一套架构产出两种相当不同的环境,也会握有设计"第三份给自己的环境"的概念工具——无论那第三份的形态长成什么样。

The founder in this case study runs a small SaaS company alone. The product is a developer tool, a paid service that helps engineering teams analyze and visualize their CI/CD pipelines. She wrote the original prototype, she runs the marketing, she handles customer support, she manages the infrastructure, and she does the bookkeeping. The company has roughly two hundred paying customers, the revenue is enough to support her full-time, and she is at the stage where her single biggest constraint is her own time. She has been a heavy Claude Code user for several months, and she has decided that her environment is overdue for serious engineering. The case study walks through the build.

本案例里的这位创业者,独自经营着一家小型 SaaS 公司。产品是一款开发者工具——一项付费服务,帮工程团队分析并可视化他们的 CI/CD 流水线。她写出了最初的原型;她跑营销;她处理客户支持;她管基础设施;她也做账。公司大约有 200 个付费客户,营收足够支撑她全职投入;她目前的阶段,最大的瓶颈就是她自己的时间。她已经重度使用 Claude Code 几个月,并认定自己的环境早该被好好"工程化"一遍。这个案例会走一遍搭建的过程。

Begin with the diagnosis. The founder's work has a distinctive shape that is worth naming explicitly, because it influences every later design decision. Her work is multi-domain. In a single working day, she might

先从"诊断"开始。这位创业者的工作有一种值得明确点出的形态——因为它会决定后面所有的设计决策。她的工作是多领域的。一个工作日里,她可能

write a product roadmap section, draft a sales email to a prospective customer, debug a production issue in the ingestion service, write a blog post about a recent feature, respond to a support ticket about API rate limits, update the company's investor dashboard, and produce next week's marketing calendar. No single role-shaped specialist would cover this range, and a generic Claude Code session that handled all of it would suffer from the context-pollution problem in an extreme form, because the residue of writing a sales email would still be in the context when she pivoted to debugging a production issue.

写一段产品 roadmap 章节;给一个潜在客户一封销售邮件;debug 一下 ingestion service 的一个生产问题;就最近一个 feature 写篇 blog;回一张关于 API rate limit 的支持工单;更新一下公司投资方看板;再做出下周的营销日历。这种范围,没有任何"单一角色"专家能覆盖;而一个把这一切都包在里面的、通用型的 Claude Code session,会以最极端的形式遭受"上下文污染"问题——因为写完销售邮件留下的那些残留,会一直在她切到 debug 生产问题时赖在上下文里不走。

Her environment, therefore, needs to be designed around clean context separation between domains. Skills will encode the conventions that apply within each domain. Subagents will provide specialist roles for the major recurring tasks. Hooks will enforce boundaries at the lifecycle moments where the founder's attention is least reliable. The directory structure will reflect the multi-domain nature of the work, with skills organized so that the right ones activate for the right kind of work without firing in inappropriate contexts.

所以,她的环境,必须围绕"领域间干净的上下文分隔"来设计。Skills 编码各领域内部适用的规约;subagents 为主要反复出现的任务提供专家角色;hooks 在生命周期里那些"她注意力最不可靠"的节点强制边界。目录结构要反映这种"多领域"的本性——skills 要组织得能让"对的 skill 在对的工作上触发,而不会在不恰当的場景下乱触发"。

The `.claude` directory at the user level (since this is a personal environment that travels across all of her projects) is organized as follows.

user 级别的 `.claude` 目录(因为这是一份跟着她走遍所有项目的"个人环境")长这样:

```
~/ .claude/
├─ settings.json
├─ skills/
│  └─ product/
│     └─ prd-section/
│     └─ feature-roadmap-entry/
│     └─ decision-log/
│  └─ marketing/
│     └─ blog-post-outline/
│     └─ feature-announcement/
│     └─ twitter-thread/
```

```
| | └─ newsletter-section/
| | └─ sales/
| |   └─ prospect-email/
| |   └─ follow-up/
| | └─ support/
| |   └─ support-reply/
| |   └─ ticket-summary/
| | └─ ops/
| |   └─ investor-update/
| |   └─ financial-summary/
| |   └─ weekly-review/
| |   └─ writing/
```

注:目录结构将在下一页继续,展示剩余的 `writing/` 、 `agents/` 、 `hooks/` 部分。

注:目录结构将在下一页继续,展示剩余的 `writing/` 、 `agents/` 、 `hooks/` 部分。

The most striking difference from the previous case study is the breadth of skills. The founder has roughly fifteen skills organized into six domain folders. The folder organization is a convention rather than a technical requirement; Claude Code does not care how skills are organized inside the skills directory. The folders exist for the founder's own benefit, so that when she opens the directory to add or edit a skill, she can find what she is looking for at a glance. The discipline of organizing skills by domain pays off as the library grows, because a flat directory with thirty skills becomes hard to navigate, while a folder-organized one stays legible indefinitely.

和上一个案例最显眼的差异,是 skills 的广度。这位创业者大致有 15 个 skill,组织在 6 个领域目录下。按目录来组织,更多是一种约定,而不是技术上的硬性要求——Claude Code 并不在意 skills 目录里怎么排。这些目录是为了她自己的便利而存在的:当她打开目录加 skill 或改 skill 时,一眼就能找到要找的东西。随着库变大,这种"按领域组织"的纪律就显出价值了——一个扁平目录里塞 30 个 skill,会变得难以导航;而按文件夹分层的可以一直保持清晰。

Begin building with the marketing domain, because it is the most distinctive of the founder's use cases. The `marketing-writer` subagent is the role specialist that handles most marketing-related tasks. It has its own system prompt, tuned for the founder's voice and the company's tone, and it loads relevant skills as it works.

从 marketing 领域开始搭起,因为它是这位创业者用法里"最不一样"的那一块。`marketing-writer` subagent 是处理大多数营销相关任务的"角色专家"。它有自己的 system prompt,针对这位创业者的口吻和公司调性调过;它在工作中会加载相关的 skill。

目录结构(续) · Directory Structure (continued)

```
| | └─ writing/
| |   └─ humanize-text/
| |   └─ shorten/
| |   └─ (minimal, see below)
| └─ agents/
|   └─ product-strategist.md
|   └─ marketing-writer.md
```



```
| |— support-helper.md
| |— financial-analyst.md
| |— code-reviewer.md
|— hooks/
|   (minimal, see below)
```

Marketing Writer Subagent · Marketing Writer subagent 定义

```
---
name: marketing-writer
description: Drafts marketing content including blog posts, feature announcements, Twitter threads, newsletter sections
tools: Read, Write, Glob
model: sonnet
---

You are the marketing voice of a small developer-tools company. The company sells a service that helps engineering t
```

注:以上是 `marketing-writer` subagent 的 YAML frontmatter 和 system prompt 开头;剩下的"你写东西的口吻"以及"方法/不做什么"两段在下一页继续。

注:以上是 `marketing-writer` subagent 的 YAML frontmatter 和 system prompt 开头;剩下的"你写东西的口吻"以及"方法/不做什么"两段在下一页继续。

Marketing Workflows · 营销 workflow

The subagent has access to the company's marketing skills, including the format-specific ones (`blog-post-outline` , `feature-announcement` , `twitter-thread` , `newsletter-section`), and the general writing skills (`humanize-text` , `shorten`). When the founder asks for a marketing piece, the `marketing-writer` subagent activates, the relevant format skill fires inside it, the `humanize-text` skill applies its pass at the end, and the result comes back as a draft in the founder's voice with the company's conventions applied.

这个 subagent 拿得到公司的 marketing skills——既包括"按形态分"的那几个(`blog-post-outline` 、 `feature-announcement` 、 `twitter-thread` 、 `newsletter-section`),也包括"通用写作"那两个(`humanize-text` 、 `shorten`)。当创业者要一段营销内容, `marketing-writer` subagent 激活;相关的"形态 skill"在它内部触发; `humanize-text` skill 在最后做一遍"去机器味"的过稿;最终回来的,是一段套用了公司规约、又带着创业者本人声音的草稿。

The `blog-post-outline` skill is small but important, because it captures the founder's preferred structure for a blog post.

`blog-post-outline` skill 不大但重要——它把这位创业者偏好的 blog 结构沉淀了下来。

Marketing Writer(续) · Marketing Writer(续)

The voice you write in is honest, technically precise, and slightly understated. You do not use marketing superlativ

Your method:

1. Activate the relevant skill for the format requested (blog post outline, feature announcement, twitter thread, ne
2. Activate the humanize-text skill at the end of every draft to remove AI-flavored vocabulary and patterns.
3. Read the existing marketing material in the company's content directory to match the voice.
4. Produce the draft in the requested format.
5. End with a short note about the angle you took and a question if you are uncertain whether the angle matches the

What you do not do:

- You do not invent product features. If the request implies a feature you have not seen documentation for, ask the
- You do not use language the founder has explicitly excluded.
- You do not produce vague marketing-speak. Every sentence should communicate something specific.
- You do not pad. A sharp short post beats a padded long one.

End with the draft and a brief note about voice or angle.

注:上面这段 system prompt 用了"口吻 + 方法 + 显式禁止"三层结构。口吻一段明确点了"禁词清单"(powerful / robust / elegant / industry-leading);方法一段规定了"先激活相关 skill、最后跑 humanize-text"的工序;显式禁止一段把"会出现的失败模式"列成可枚举的负面条款。

注:上面这段 system prompt 用了"口吻 + 方法 + 显式禁止"三层结构。口吻一段明确点了"禁词清单"(powerful / robust / elegant / industry-leading);方法一段规定了"先激活相关 skill、最后跑 humanize-text"的工序;显式禁止一段把"会出现的失败模式"列成可枚举的负面条款。

← 前一页

后一页 →

Move to the support domain. The `support-helper` subagent handles customer support tasks, with the `support-reply` skill providing the format and the `ticket-summary` skill providing a way to compress long ticket threads into actionable summaries.

接下去是 support 领域。`support-helper` subagent 处理客户支持任务——`support-reply` skill 提供格式, `ticket-summary` skill 提供把长 ticket 线程压缩成可行动摘要的能力。

Blog Post Outline Skill · Blog Post Outline skill 定义

```
---
name: blog-post-outline
description: Produces a blog post outline in the company's standard structure. Use whenever the user asks for a blog
allowed-tools: Read
---
# Blog Post Outline

When asked for a blog post, structure it as follows.

1. Hook (one paragraph): Open with a specific, concrete observation or question. Avoid abstract framing. The hoo
2. The problem (one or two paragraphs): Describe the actual pain the post will address, in technical terms. Use
3. The mental model or framework (two or three paragraphs): Lay out the way of thinking about the problem that t
4. The application (one or two examples): Show the framework applied to one or two concrete cases. Use real code
5. The trade-offs (one paragraph): Be honest about what the framework does not solve, where it does not apply, a
6. The takeaway (one or two sentences): A single concrete statement of what the reader should do or remember.
```

Length target: 1,200 to 1,800 words. Longer is acceptable when the topic justifies it. Shorter is acceptable when the topic is narrow.

After producing the outline, ask the founder to confirm the angle before drafting the full post.

Support Helper Subagent · Support Helper subagent 定义

```
---
name: support-helper
description: Drafts customer support replies and summarizes long support threads. Use whenever the user mentions a customer support request.
tools: Read, Write
model: sonnet
---

You are a senior customer support engineer for a small developer-tools company. The customers are technical: engineers and product managers. You are helpful, concise, and clear. You are not afraid to ask for clarification if the user's request is ambiguous. You are not afraid to ask for clarification if the user's request is ambiguous. You are not afraid to ask for clarification if the user's request is ambiguous.
```

注:以上是 support-helper subagent 的 YAML frontmatter 和 system prompt 开头;剩下的"口吻 / 方法 / 不做什么"段在下一页继续。

注:以上是 support-helper subagent 的 YAML frontmatter 和 system prompt 开头;剩下的"口吻 / 方法 / 不做什么"段在下一页继续。

Move to the operations domain. The financial-analyst subagent handles tasks like generating investor updates, summarizing the company's monthly financials, and producing weekly review reports.

接下去是 operations 领域。financial-analyst subagent 处理诸如"投资方更新生成、公司月度财务摘要、周报"等任务。

Support Helper(续) · Support Helper(续)

```
and substantive. You do not use phrases like "I completely understand your frustration." You acknowledge the issue and express empathy. You do not use phrases like "I completely understand your frustration." You acknowledge the issue and express empathy. You do not use phrases like "I completely understand your frustration." You acknowledge the issue and express empathy.

Your method:

1. Read the customer's message carefully. Identify what they actually need: a fix, an explanation, a workaround, an apology, or a combination of these.
2. If the answer involves product knowledge you are not sure about, flag the uncertainty rather than guess. The founder is your best resource for this.
3. Activate the support-reply skill for format guidance.
4. Draft the reply.
5. If the issue is a bug or a known limitation, end with a sentence that indicates whether it is on the roadmap to fix.

What you do not do:
- You do not promise fixes you cannot guarantee.
- You do not use customer service platitudes.
- You do not pad. Customers' time is valuable.
- You do not ignore technical specifics. If the customer mentions a version number, an error code, or a configuration, you must include it in the reply.

End with the draft reply and a one-line note about whether you are confident in the technical content or whether the issue is a bug or a known limitation.
```

Financial Analyst Subagent · Financial Analyst subagent 定义

```
---
name: financial-analyst
description: Produces investor updates, monthly financial summaries, and weekly business reviews using data from the
tools: Read
model: opus
---
```

You are a senior financial analyst working with a small SaaS founder. Your job is to take raw metrics and accounting

Your principles:

- Numbers without context are not useful. Every metric should be paired with a comparison: month-over-month, year-over-year

注:以上是 `financial-analyst` subagent 的 YAML frontmatter 和 system prompt 开头;剩下的"原则 / 方法 / 不做什么"在下一页继续。注意它选的是 `model: opus` ——这位创业者在"涉及数字精度"的任务上,愿意多花 token 换更稳的判断。

注:以上是 `financial-analyst` subagent 的 YAML frontmatter 和 system prompt 开头;剩下的"原则 / 方法 / 不做什么"在下一页继续。注意它选的是 `model: opus` ——这位创业者在"涉及数字精度"的任务上,愿意多花 token 换更稳的判断。

← 前一页

后一页 →

The other subagents in this environment include `product-strategist`, which handles product roadmap and PRD work, and `code-reviewer`, which is a near-copy of the engineering-focused subagent for the times when the founder is reviewing her own code.

本环境里其他的 subagent 包括 `product-strategist` ——它处理产品 roadmap 和 PRD 相关工作;以及 `code-reviewer` ——它是"工程向 subagent 的一个近复制版",在创业者 review 自己代码时出场。

Move now to the hooks. The hooks for this environment are deliberately minimal, because the founder's work does not have the same enforcement requirements as the team-engineering project from the previous chapter. Her work is largely solo, the consequences of mistakes are concentrated on her own time rather than on a team's infrastructure, and over-engineering the hook layer would produce friction without proportionate benefit. Her `settings.json` contains only three hooks.

接下来是 hooks。这个环境的 hooks 是有意做得极简的——因为这位创业者的工作并不像上一章的"团队工程"那样有同等的强制需求。她基本是单干;错误的后果集中在她自己的时间上,而不是团队的基础设施上;过度工程化 hook 层只会带来和收益不成比例的摩擦。她的 `settings.json` 里只有 3 个 hook。

Financial Analyst(续) · Financial Analyst(续)

- Be honest about bad numbers. A report that hides a slowdown is a report that delays the founder's response to a problem.
- Be specific about what is changing. "Revenue grew" is not a useful sentence. "Revenue grew 8% MoM, driven by three new products" is.
- Distinguish leading indicators from lagging indicators. New trial signups are leading. MRR is lagging. Both matter.

Your method:

1. Activate the relevant format skill (investor-update, financial-summary, weekly-review).
2. Read the data sources the founder has pointed you at.
3. Produce the report in the format the skill specifies.

4. End with a short section labeled "Things I would watch," where you flag anything in the data that you think deserves

What you do not do:

- You do not invent numbers. If a number is not in the data, you do not include it.
- You do not editorialize beyond the "things I would watch" section.
- You do not produce reports longer than necessary. Density beats length.

End with the report and the "things I would watch" section.

Settings.json(节选) · Settings.json(节选)

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "$HOME/.claude/hooks/block-dangerous.sh"
          }
        ]
      }
    ]
  }
}
```

注:这段是 settings.json 的开头;剩下的 Edit|Write|MultiEdit 门和 Notification 通知 hook 将在下一页继续。

注:这段是 settings.json 的开头;剩下的 Edit|Write|MultiEdit 门和 Notification 通知 hook 将在下一页继续。

The first hook blocks dangerous shell commands. The second protects sensitive file paths, including the founder's .env files, her accounting exports, and her investor communications archive. The third produces a desktop notification whenever Claude needs her attention, because she often has Claude Code running in a background terminal while she works on something else and would otherwise miss the prompt for input.

第一个 hook 用来拦危险的 shell 命令。第二个保护敏感文件路径——包括她的 .env 文件、她的会计导出、她的投方沟通存档。第三个在 Claude 需要她注意时弹一条桌面通知,因为她经常把 Claude Code 挂在后台 terminal 里,自己同时在干别的事,如果不弹通知,她很容易错过 Claude 抛过来的提问。

A Minimal Hook Layer · 极简的 hook 层

The minimalism of the hook layer is a design choice, not an oversight. The founder considered adding test-running hooks, formatting hooks, and several other guards from the previous case study, but concluded that the friction of those hooks would outweigh their benefit in her solo, multi-domain context. Her work moves between writing, code, and analysis throughout the day, and a hook that runs the test suite after every Edit would slow down a writing session where she is editing markdown files that have nothing to do with code. The right call for her was to keep the

Hook 层的极简,是一个设计选择,不是疏漏。这位创业者考虑过加"跑测试的 hook""格式化的 hook"以及上一章里的另外几道门;但她最后判断:在她的"单干、多领域"语境下,这些 hook 带来的摩擦大于收益。她一天里在写作、代码、分析之间来回切换;一个"每次 Edit 之后跑测试套件"的 hook,会拖慢一次"她在改完全跟代码无关的 markdown 文档"的写作 session。对她来说,正确的选择是——

```

    }
  ]
},
{
  "matcher": "Edit|Write|MultiEdit",
  "hooks": [
    {
      "type": "command",
      "command": "$HOME/.claude/hooks/protect-files.sh"
    }
  ]
}
],
"Notification": [
  {
    "hooks": [
      {
        "type": "command",
        "command": "osascript -e 'display notification \"Claude needs your input\" with title \"Claude Code\"'"
      }
    ]
  }
]
}
}
}

```

注:这段 `settings.json` 比上一章那份明显瘦一圈——只有 1 个 Bash 门(拦危险命令)、1 个文件门(保护 `.env` /会计导出/投方存档)、再加 1 个 macOS 桌面通知(`osascript -e 'display notification ...'`)。这种"够用就好"的选择,正是为单干多领域的工作量身定的。

注:这段 `settings.json` 比上一章那份明显瘦一圈——只有 1 个 Bash 门(拦危险命令)、1 个文件门(保护 `.env` /会计导出/投方存档)、再加 1 个 macOS 桌面通知(`osascript -e 'display notification ...'`)。这种"够用就好"的选择,正是为单干多领域的工作量身定的。

[< 前一页](#)
[后一页 >](#)

hook layer light and let the discipline live in her skills and subagents instead. A different founder, with a different shape of work, might make different choices. The architecture accommodates both.

把 hook 层保持轻量,让"规约纪律"由 skills 和 subagents 去承担。一位形态不同的创业者,可能做出不同的取舍。这套架构两边都容得下。

A walk through a representative day in this environment will make the difference visible. The founder opens Claude Code in the morning. She has a busy day planned: a customer support reply, a section of next month's investor update, a marketing post about a feature she shipped last week, and some debugging of a production issue she noticed in last night's metrics.

走一遍这个环境里"代表性的一天",可以让这种差别可见。创业者早上打开 Claude Code。今天安排得很满:一封客户支持回复、下月投资方更新里的一节、上周发布的那个 feature 的 marketing post、再加上昨晚 metrics 上注意到的一个生产问题,要 debug 一下。

She starts with the support reply. She reads the ticket, then types into Claude Code, draft a reply to this customer about the rate-limiting issue. The `support-helper` subagent activates because the request matches its description. The subagent reads the customer's message, drafts a reply that acknowledges the issue concretely and offers a workaround, and ends with a note that the rate-limit configuration change the customer is asking about is on the roadmap for the next release. She reviews the draft, makes a small wording change, and sends it.

她从支持回复开始。她读了 ticket,然后在 Claude Code 里敲:给这个客户草一份关于 rate-limiting 问题的回复。`support-helper` subagent 因为请求匹配上 description 而激活。Subagent 读客户消息,草出一份"具体承认问题 + 提供临时方案"的回复,末尾加一句:客户想要的 rate-limit 配置调整,已经排在下一版的 roadmap 上。她 review 草稿,改了一两个词,发出去了。

She moves to the investor update. She types, write the metrics section of this month's investor update; data is in `/investors/data/january.csv`. The `financial-analyst` subagent activates. It reads the data, produces a section that opens with the headline metric, follows with the MoM and YoY comparisons, breaks down the drivers, and ends with the "things I would watch" section that flags an unusual dip in trial-to-paid conversion in the second week of the month. She thanks the analyst for the catch, makes a note to investigate, and copies the section into her investor update document.

她接着做投资方更新。她敲:写本月投资方更新的 metrics 章节;数据在 `/investors/data/january.csv`。`financial-analyst` subagent 激活。它读数据,生成一节内容:头条指标打头,接着是 MoM 和 YoY 对比,然后拆解 driver,最后是"things I would watch"小节——点出本月第二周一次不寻常的"试用转付费"下滑。她感谢这个 subagent 抓到了这条,记下来要查一下,然后把这一节粘进投资方更新文档。

She moves to the marketing post. She types, draft a blog post about the new pipeline visualization feature we shipped last week. The `marketing-writer` subagent activates. The `blog-post-outline` skill fires inside it. The subagent produces an outline following the company's standard structure, asks her to confirm the angle, she confirms, and the subagent drafts the full post. The `humanize-text` skill applies its pass at the end, removing AI-flavored vocabulary and tightening the prose. She reads the draft, finds it about ninety percent of where she wants it, makes a few specific edits, and schedules it for publication.

接着是 marketing post。她敲:给上周发布的那个新的 pipeline 可视化 feature 草一篇 blog。`marketing-writer` subagent 激活。`blog-post-outline` skill 在它内部触发。Subagent 按公司标准结构出一份大纲,问她确认"切入点",她确认,然后 subagent 写完整篇。`humanize-text` skill 在最后跑一遍,去掉 AI 味、收紧句子。她读一遍,觉得约九成到位,改了几处,排期发布。

← 前一页

后一页 →

She moves to the production debugging. The production code lives in a separate project directory, and she opens Claude Code there. The project-level configuration in that directory is much closer to the engineering case study from the previous chapter, with skills, subagents, and hooks tuned for the engineering work. The founder's personal user-level configuration from this chapter still applies, but the project-level configuration extends it with engineering-specific tooling. She debugs the production issue, fixes it, gets a code review from her `code-reviewer` subagent, and ships the fix. The whole cycle takes thirty minutes.

她接着处理生产 debug。生产代码住在另一个项目目录里,她在那里打开 Claude Code。那个目录的 project-level 配置,和上一章那个工程案例非常接近——有针对工程工作调好的 skills、subagents、hooks。本章讨论的"user 级别"个人配置仍然生效,但 project-level 配置在它之上又"接"了一层专门给工程用的工具。她 debug 完生产问题、改好、请 code-reviewer subagent 给她做一轮 review,然后把修复推上去。整个闭环 30 分钟。

The total time across the four tasks is roughly two and a half hours. Without the environment, the same four tasks would have taken her closer to five hours, and several of them would have come out worse. The investor update would have lacked the "things I would watch" framing. The marketing post would have sounded like every other AI-drafted marketing post. The support reply would have either been too long or too short, depending on her mood. The environment did not make her smarter. It made her output more consistent and her time more efficient, by absorbing the conventions, the formats, and the role-shaping that her work requires.

四件事加起来的总时间,差不多两个半小时。没有这个环境的话,同样的四件事会更接近五个小时;而且其中几件出来的质量会更差。投方更新会缺那层"things I would watch"的框架;marketing post 会听起来跟所有别的"AI 起草"营销文一样;支持回复——视她当时心情——要么过长、要么过短。环境没让她变聪明——环境通过吸收她工作里那些"规约、格式、角色塑形"的部分,让她的产出更一致、她的时间更高效。

Generalizing Beyond This Case · 跨出本案例去推广

A reflection on what this case study demonstrates relative to the previous one. The two environments are strikingly different in surface appearance. The full-stack engineering environment has eight skills focused on engineering tasks, four engineering specialists, and six enforcement hooks. The solo founder environment has fifteen skills spanning six domains, five role specialists across product, marketing, support, ops, and engineering, and three minimal hooks. The two environments could not look less alike on first glance. But the underlying architecture is identical. Both use the three-pillar framework. Both organize knowledge into skills, roles into subagents, and boundaries into hooks. Both compose the three pillars to produce capabilities that no single pillar could provide. The framework is what makes the two environments coherent, even though their content is entirely different.

先回头看一下:相对于上一个案例,这个案例究竟说明了什么。两个环境在"表面长相"上的差距是惊人的。全栈工程环境有 8 个 skills 全部面向工程任务、4 个工程方面的专家、6 个强制 hook。独立创业者环境有跨 6 个领域的 15 个 skills、跨产品/营销/支持/运营/工程 5 个角色专家、3 个极简 hook。两者乍一眼看上去,没有比这更不像的了。但底下的架构是同一套。两者都用了三柱框架;都把知识组织进 skills、角色塑形进 subagents、边界强制进 hooks;都通过三柱组合产出"任何一柱单干都拿不出的"能力。框架,正是让这两份环境在"内容天差地别"的前提下仍保持连贯的那条线。

A second reflection concerns the generalizability of the framework. The two case studies in this part of the book are deliberately different to make the generalizability point. A reader who works in a third shape of work,

第二条反思是关于"框架的可推广性"。本书这一部分的两个案例,故意做得不一样,就是为了把"可推广"这一点立起来。一位读者,如果从事的是第三种形态的工作——既不是全栈工程,也不是独立 SaaS 创业——

neither full-stack engineering nor solo SaaS founding, can take the framework and apply it to her own context. The classification questions still apply: knowledge gap means skill, attention gap means subagent, guarantee gap means hook. The composition principle still applies: skills inside subagents, subagents in parallel orchestration, hooks at the lifecycle boundaries. The maintenance discipline still applies: start small, add components when their need is concrete, remove components that have not earned their place. The reader who has read this book carefully has a complete blueprint for engineering her own Claude Code environment, regardless of what kind of work she does.

也可以拿这套框架,套到自己的语境上。分类问题依然成立:知识缺口→skill,注意力缺口→subagent,保证缺口→hook。组合原则依然成立:skills 嵌入 subagents、subagents 并行编排、hooks 守在生命周期边界。维护纪律依然成立:起步要小、需求具体时才加、用得不够就清。认真读过本书的读者,手里已经握有一份"为自己工作工程化一份 Claude Code 环境"的完整蓝图,不管她的工作是哪一种。

A third reflection concerns the founder's specific situation as illustrative of a broader pattern. The founder in this case study is the entire labor force of her company. Her time is the binding constraint. The Claude Code environment is, in her case, not just a productivity improvement; it is the difference between a one-person company that can sustain its operations and one that cannot. The leverage that the environment provides is not optional for her. It is the precondition for the company existing at all in its current form. A reader who is also operating at this kind of scarce-time, multi-domain shape of work should recognize her own situation in the case study and recognize that the architecture this book has been teaching is, for her, a strategic asset rather than a developer convenience.

第三条反思,是关于"这位创业者的具体处境作为更广范式的例证"。本案例里的创业者就是她公司的全部劳动力——她的时间是真正的紧约束。对她来说,Claude Code 环境不只是"生产力改进",它是有和没有之间——一家"一个人撑着、且能维持运转"的公司 vs 一家"维持不了"的公司。环境提供给她的杠杆,对她而言不是"可选"——它是"公司以当前形态存在"这件事本身的前提条件。一位同样处于"时间稀缺、多领域"这种工作形状的读者,应当在这个案例里看到自己的处境,看到本书一直在教的这套架构——对她来说——是战略性资产,而不是开发者的便利。

A second worked walkthrough is worth including, because the Friday weekly-review session exercises a different combination of the founder's environment than the daily-task scenario above and shows the architecture from another angle.

第二个落地演示也值得放进案例。"周五周会回顾"这个 session,会锻炼到上面那个"日常任务"场景没有锻炼到的那一组环境组合,可以从另一个角度把这套架构展示出来。

It is Friday afternoon. The founder sits down to do her weekly review, which she does every Friday before she logs off for the weekend. The review has three components: a financial weekly summary, a writeup of what shipped during the week, and a planning section for the following week. She opens Claude Code in the directory where she keeps her business operations files. She types, run my weekly review.

周五下午。这位创业者坐下来做她的"周会回顾"——她每个周五、在周末下线前都会做一次。这次回顾包含三块:一份财务周摘要、一段本周已交付的总结、以及下一周的规划。她在"业务运营文件"所在的目录里打开 Claude Code,敲:跑一遍我的周会回顾。

The financial-analyst subagent activates first, because the weekly-review skill it is paired with carries the highest priority in the matching. The subagent reads the metrics dashboard export from earlier that day, reads the previous week's summary for comparison, and produces the financial

financial-analyst subagent 首先被激活——因为和它配对的 weekly-review skill,在匹配里拥有最高优先级。Subagent 读当天早些时候导出的 metrics dashboard、读上一周的周报做对比,然后产出"财务"那块——

section: revenue change, customer count change, churn analysis, and the "things I would watch" section. The "things I would watch" section flags an unusual cluster of support tickets all related to the same API endpoint. The founder makes a mental note.

营收变化、客户数变化、churn 分析、再加"things I would watch"小节。这最后这一节点出了一组不寻常的、与同一 API endpoint 相关的支持工单。这位创业者心里记了一笔。

The `marketing-writer` subagent activates next, because the founder also wants the weekly review to capture what shipped on the marketing side. The subagent reads the blog posts published that week, the social media posts, the newsletter that went out, and produces a short summary of marketing output with attached engagement metrics where available.

接着轮到 `marketing-writer` subagent 激活——因为这位创业者也希望周报把"marketing 这一侧这周发了什么"记下来。Subagent 读当周发布的 blog posts、社交媒体上的帖子、发出去的 newsletter,产出一段"marketing 产出"的短摘要,能附上 engagement metrics 的地方都附上。

The Weekly Review · 周会回顾

The `product-strategist` subagent then handles the planning section. It reads the founder's product roadmap document, reads the recent decision logs from the product folder, and produces a candidate list of three to five things to consider tackling next week, each with a one-sentence rationale.

然后是 `product-strategist` subagent 接手"规划"那一节。它读这位创业者的产品 roadmap 文档、读 product 文件夹里最近的决策日志,产出 3~5 件"下周可以考虑推进"的候选清单,每件附一句"为什么值得做"的理据。

The three sections come together as a single document. The founder reads it, edits a few wording choices in the marketing section, expands the rationale for one of the planning items, and saves it to her ops folder. The whole review takes twenty minutes, where it would have taken closer to ninety without the environment, and the resulting review is more disciplined and more honest than the ones she used to write by hand, because the `financial-analyst` subagent does not let her gloss over uncomfortable numbers.

三节合起来,成了一份单一文档。她通读一遍,在 marketing 那节改了几处措辞,把"规划清单"里某一条的理据展开了一句,然后存到她的 ops 文件夹里。整个回顾 20 分钟;没有这个环境会更接近 90 分钟。而且出来的这份周报比她过去手写时更守纪律、更诚实——因为 `financial-analyst` subagent 不会让她糊弄那些"看着不舒服"的数字。

The cluster of support tickets the `financial-analyst` flagged turns out to be the most consequential output of the review. She investigates the cluster, finds that the affected endpoint has a subtle race condition under high concurrent load, and ships a fix that following Tuesday. Without the `financial-analyst`'s "things I would watch" framing, the cluster would have been buried in the support volume and probably not investigated for several more weeks, by which point a few of the affected customers might well have churned. The investment in the `financial-analyst` subagent has, in effect, paid for the entire environment in this single session, by surfacing a pattern that human attention alone might have missed.

被 `financial-analyst` 点出来的那组支持工单,最后成了整份周报里"最值钱的产出"。她追查了这组工单,发现受影响的 endpoint 在高并发下有一个微妙的竞态;她下了个修复,在接下来的那个周二推上线。如果没有 `financial-analyst` 那条"things I would watch"框架,这一组工单大概会被埋在支持工单的整体量里、几周之内都不会被仔细看;等到几周之后才被发现时,几位受影响的客户很可能已经流失了。为 `financial-analyst` subagent 投入的那些功夫,事实上就靠这一次的发现,就把整个环境的成本收回来了——它把"光靠人的注意力很可能漏掉"的那一类模式,主动浮到了水面上。

The example shows the multi-domain nature of the founder's environment functioning at full strength. Three subagents from three

这个例子,把这位创业者的环境"多领域"的本性,完整地展示了出来。来自三个不同领域的 3 个 subagent——

← 前一页

后一页 →

different domains coordinated on a single integrated task. Each contributed its specialty. The composition produced an output that exceeded what any single subagent could have produced. The founder's role in the work shifted from drafting all three sections by hand to reviewing and editing three drafts, which is a higher-leverage form of attention. This is what the architecture is supposed to produce: not the elimination of the founder from the loop, but the elevation of her contribution from drafting to direction.

协调地跑同一件综合性任务。每一个都贡献自己的专长;组合起来产出的成果,超出了任何单一 subagent 单独能干的事。这位创业者在其中的角色,从"自己手写三节"转变为"读三份草稿并修订"——这才是更高杠杆形态的注意力。这正是这套架构想要做的事:不是把创业者从循环里拿掉,而是把她的贡献从"亲笔起草"提升到"把握方向"。

A fourth reflection concerns the limits of what this kind of environment can do. The architecture does not make the founder smarter. It does not produce ideas she would not have produced. It does not make strategic decisions for her. The architecture absorbs the conventions, the formats, and the role-shaping that her work requires, leaving her attention free for the work that actually requires her judgment. The strategic decisions, the brand voice that gives the marketing its character, the product instincts that decide which feature to build next, all of those remain hers. The environment is a tool, and like all tools, it amplifies what is already there rather than replacing it. A founder with poor strategic instincts will not become a great strategist by installing a Claude Code environment. A founder with strong instincts will find that her instincts now reach further, because the surrounding labor of executing on them is partly absorbed by the environment.

第四条反思是关于"这类环境能做到的边界"。这套架构不会让创业者变聪明——不会替她生出她本来没有的创意,也不会替她做战略决策。它吸收她工作里那些"规约、格式、角色塑形"的部分,把她的注意力腾出来,留给那些真正需要她判断的事。战略决策、构成 marketing 性格的品牌调性、决定下一个该做哪个 feature 的产品直觉——这些都还是她的事。环境是工具;工具和所有工具一样,放大它已经在那里的东西,而不会去替代它。一位战略直觉差的创业者,装一套 Claude Code 环境不会让她变成伟大的战略家。一位直觉强的创业者,会发现自己的直觉能到达更远的地方——因为执行那条直觉所围绕的那些"周边劳动",有一部分被环境吸收了。

A fifth reflection concerns the readiness of this kind of environment for evolution. The founder's environment will not stay the same. As her business grows, the work will shift. She might hire her first employee, at which point some of the personal user-level configuration will need to be migrated to project-level so that the new hire inherits the relevant pieces. She might launch a second product line, at which point new domain folders will appear in her skills directory. She might shift from monthly to weekly investor updates, at which point the `financial-analyst` subagent will need its description and its system prompt updated. The architecture accommodates all of these evolutions naturally, because each component is independently editable and the relationships between components are loose enough to evolve gradually. The environment that the

founder has at the end of her first year will not look identical to the environment at the start of the year, and the difference will be a record of how her work has evolved. This is the deeper meaning of treating the `.claude` directory as a living artifact: the

第五条反思是关于"这类环境对演进的准备度"。这位创业者的环境不会一成不变。她的业务一旦长大,工作本身就会变。她可能会招第一名员工——那时一部分 `user` 级别的个人配置就需要迁到 `project` 级别,让新同事能继承到对应的那部分。她可能会开第二条产品线——那时她的 `skills` 目录里会出现新的领域文件夹。她可能会把"投方更新"从月度改成周度——那时 `financial-analyst` subagent 的 `description` 和 `system prompt` 都要更新。这套架构对所有这些演化都是天然友好的——因为每一个组件都可以独立编辑,组件之间的关系也足够松,可以逐步演化。这位创业者在第一年年末手里那份环境,看上去不会和年初那份一样;而这两份之间的差异,本身就是她工作演化的记录。这正是"把 `.claude` 目录当作活着的工件"更深的含义:这个目录——

← 前一页

后一页 →

directory becomes a mirror of the work, and the work becomes more legible by being mirrored.

变成了一份工作的镜子;而被镜照过之后,工作本身也变得更可读了。

A sixth and final reflection concerns the broader implication of multi-domain Claude Code environments for the way solo founders and small teams operate in the present moment. The combination of cheap intelligence and well-engineered environments has shifted the boundary of what a single person can sustain operationally. Companies that would have required a team of five a decade ago can now be run by a single founder with a thoughtful Claude Code environment, because the environment absorbs the operational labor that used to require additional headcount. The leverage is real and substantial. Founders who recognize this shift and invest in their tooling accordingly are operating with a structural advantage over founders who treat their tools as commodities. The architecture this book has been teaching is, in this sense, a piece of strategic infrastructure for solo and small-team operators in the current era, and the case study above is meant to demonstrate not merely a productivity gain but a different kind of company that becomes possible to run when the architecture is in place.

第六条也是最后一条反思,是关于"多领域 Claude Code 环境"对"独立创业者和小型团队当下如何运作"这件事更广泛的意义。"廉价的人工智能 + 工程化良好的环境"这一组合,已经移动了"一个人能在运营上撑住多大规模"那条边界。十年前需要一个 5 人团队才能撑起来的公司,如今可以由一位独立创业者 + 一份经过深思熟虑的 Claude Code 环境撑起来——因为环境吸收了过去需要额外人手才能消化的运营劳动。这种杠杆是真实且可观的。看清这一变化、并据此投资自己工具的创业者,比起把工具当大宗商品对待的创业者,具备一种结构性的优势。本书一直在教的架构,在这个意义上,是当下"独立 / 小型团队"运营者手头的一块战略性基础设施;上面这个案例,想说明的也不仅是一份"生产力收益",而是一种"在架构到位之后才成为可能"的、不一样的公司形态。

Part Five ends here. The architecture has been laid out, and two case studies have shown it producing complete environments for very different shapes of work. The reader has, by this point in the book, both the conceptual framework and the practical patterns to build her own environment for her own work. The remaining chapter of the book takes the focus from building the environment to evolving it over time, sharing it with others, and fitting it into the broader Claude Code ecosystem that is forming around the framework. That closing work is the subject of the chapter ahead.

第五部分到此收束。架构已经铺开;两份案例已经演示了它为形态迥异的工作产出完整环境。到本书这里,读者手里已经既有了概念框架,也积累了实践范式,可以为"自己的那份工作"搭出对应的环境。剩下的一章,焦点从"搭"转到"演化"——随时间推移怎么长、和他人怎么分享、以及怎么接入"围绕这套框架正在形成"的更广 Claude Code 生态。这份收尾的工作,就是下一章的主题。

